THEORY OF COMPUTATION

Project Report

GrammarGuard

Provably Valid Language-Model Output through Finite Automata and Pushdown Automata, with Formal Tools for Validator Analysis

Submitted by

Mankirat Singh Bhatia (24BIT0048)

Lakshit Bhatia (24BIT0012)

October 2026

Source code: https://github.com/MankiratBhatia/Grammar-Guard

Table of Contents

1. Aim of the Project
2. Project Title
3. Abstract
4. Register Numbers
5. Students' Names
6. Tools Used
7. Highlights and Features
8. Project Modules
9. Module Description
    9.1 M1: Regular-expression front end and automata construction
    9.2 M2: Closure operations, constraint algebra and DFA-to-regex
    9.3 M3: Language analysis and validator testing
    9.4 M4: Context-free side: grammars, LL(1), pushdown automaton and schema compiler
    9.5 M5: Language-model decoding engine and certificates
    9.6 M6: Backend service (FastAPI)
    9.7 M7: Web frontend (React and TypeScript)
    9.8 M8: Evaluation, command line, quality and packaging
10. Role of Team Members
11. Software Description
    11.1 Overview
    11.2 System requirements and installation
    11.3 Functional description
    11.4 Application programming interface
    11.5 Quality assurance
    11.6 Limitations
12. Coding
13. Screenshots of the Errors Segments
    Part A. Errors reported by the application
    Part B. Errors met during development
14. Screenshots of the Error Recovered Segments
    Part A. Recovery in the application
    Part B. Corrections made during development
15. Possible Future Work
16. Conclusion
17. References

1. Aim of the Project

Large language models write text one small piece at a time and cannot be relied on to follow a required format. A program that needs a date such as `2024-05-17`, a phone number, an employee ID or a valid JSON document cannot trust a model that usually, but not always, complies.

The aim of this project is to **guarantee, by construction, that the text a language model produces satisfies a formal specification**, and to do so by implementing the machinery of the Theory of Computation course from scratch rather than calling an existing library.

The specific objectives were:

1. To convert a regular expression into a finite automaton using Thompson's construction, subset construction and minimization, with every algorithm written by the team using only the Python standard library.
2. To connect that automaton to a real language model (GPT-2) so that, at every step, only tokens that keep the text completable to a valid string are allowed, and the model may stop only in an accepting state.
3. To extend the guarantee to nested structures (JSON) with a deterministic pushdown automaton built from an LL(1) grammar.
4. To use the closure properties of regular languages (product, complement, concatenation, star), the equivalence algorithm and the DFA-to-regex conversion as practical tools: combining rules, comparing two versions of a validator with shortest counterexamples, and generating test suites automatically.
5. To make the algorithms visible and checkable: step-by-step players for subset construction and table filling, and verifiable certificates of membership.
6. To measure the benefit objectively with a benchmark that compares the method against fair baselines, with confidence intervals and an independent correctness check.

2. Project Title

GrammarGuard: Provably Valid Language-Model Output through Finite Automata and Pushdown Automata, with Formal Tools for Validator Analysis

3. Abstract

Language models such as GPT-2 generate text token by token and often ignore the format a downstream program requires. This project presents **GrammarGuard**, a system that makes a model's output provably match a formal specification. A regular expression (or a combination of rules) is parsed, converted to an epsilon-NFA by Thompson's construction [7], determinized by subset construction [8] and minimized by table filling based on the Myhill-Nerode theorem [10, 11]. Each of the roughly 50,000 tokens of the model's vocabulary is then run through the resulting DFA to learn, for every live state, which tokens keep the output completable; at generation time all other tokens are removed before the model chooses, and the end-of-text token is allowed only in an accepting state. This follows the automaton-over-vocabulary approach of [1] and [3], implemented here with a vocabulary trie that prunes dead branches and a lazy index for large automata. A deterministic pushdown automaton driven by an LL(1) table [14, 15] extends the guarantee to JSON, optionally guided by a user-supplied schema.

Because the automata are implemented in the project itself, they also support tools that generic decoders lack: a constraint algebra built on closure properties, equivalence checking with shortest counterexamples, DFA-to-regex conversion [9], exact counting and uniform sampling of a language, automatic test-suite synthesis with full transition coverage, and certificates that anyone can re-verify without the model. A per-token metric, *constraint pressure*, measures in bits how strongly the specification overrode the model; it equals the Kullback-Leibler divergence [24] between the constrained and the unconstrained distribution.

The system is delivered as a FastAPI service with 17 operations and a React web application with seven pages. In a benchmark of 288 runs over 18 realistic validators (dates, phone numbers, identifiers, e-mail addresses and combined policies), plain GPT-2 produced an output matching the validator in 0% of runs (11% if a valid prefix is extracted from its output) and best-of-five retrying in 3% (17% with extraction), whereas GrammarGuard succeeded in 100% of runs (95% interval 97-100%), at 1.34 times the per-token cost. All regex outputs were cross-checked with Python's own `re` module with no disagreement. The software comprises about 7,450 lines of source in eight modules, is covered by 401 Python and 34 frontend tests, and is available as open source.

4. Register Numbers

Student	Register number
Mankirat Singh Bhatia	24BIT0048
Lakshit Bhatia	24BIT0012

5. Students' Names

S. No.	Name	Register number
1	Mankirat Singh Bhatia	24BIT0048
2	Lakshit Bhatia	24BIT0012

6. Tools Used

Versions are those installed in the development environment (Ubuntu on Windows Subsystem for Linux 2, 12 CPU cores, 8 GB RAM, no GPU).

Category	Tool	Version	Purpose in the project
Language	Python	3.11.17	All backend, theory and decoding code
Theory core	Python standard library only	-	Regex parser, NFA, DFA, minimization, closure, grammar and PDA code; no automata or regex library, and `re` is used only in tests as an oracle
Language model	GPT-2 (124 M parameters), pretrained	-	Model that is constrained [19]
Deep-learning stack	PyTorch (CPU build)	2.14.1	Tensor operations for the model and for masking logits
	Hugging Face Transformers	5.21.0	Loads GPT-2 and its tokenizer [22]
Backend	FastAPI	0.162.0	HTTP API, OpenAPI documentation
	Uvicorn	0.54.0	ASGI server
	Pydantic / pydantic-settings	2.14.1 / 2.16.2	Request and response models; configuration from environment variables
	sse-starlette	3.6.0	Server-Sent Events for token streaming
Frontend	React	18.3.1	User interface
	TypeScript	5.9.3	Strictly typed frontend code; types generated from the backend schema
	Vite	8.3.5	Build tool and development server
	Tailwind CSS	4.3.3	Styling, light and dark themes
	Viz.js (Graphviz in WebAssembly)	3.31.0	Draws automata in the browser [27]
Testing and quality	pytest / Hypothesis	9.2.0 / 6.170.12	Unit and property-based tests [25]
	Vitest and Testing Library	4.1.12	Frontend tests
	ruff / mypy	0.17.3 / 2.4.0	Linting; static type checking of the theory core
Runtime	Node.js / npm	20.20.2 / 10.8.2	Frontend toolchain
Delivery	Git and GitHub; GitHub Actions	-	Version control, continuous integration
	Docker (Dockerfile and compose file written)	-	Container packaging; the image was not built on the development machine
Environment	Ubuntu on WSL2; Visual Studio Code; Google Chrome (headless)	-	Development, and capture of the screenshots in this report
AI assistance	Claude Code (AI coding assistant)	-	Used during development for code generation, review and debugging; disclosed here for transparency

7. Highlights and Features

- **Guaranteed format.** The output of GPT-2 always matches the specification: every allowed token keeps the automaton in a *live* state, and end-of-text is permitted only in an accepting state. In 108 of 108 benchmark runs the output was valid.
- **Everything from scratch.** Regex parser, Thompson's construction, epsilon-closure, subset construction, two minimization algorithms (table filling and Moore's partition refinement), product, complement, concatenation, star, equivalence, DFA-to-regex, FIRST/FOLLOW, LL(1) table and pushdown automaton are all implemented in the project with only the Python standard library.
- **Constraint algebra.** Rules can be combined with and, or, minus, not, `length`, `contains`, `starts`, `ends`, `concat` and `star`; the page shows each closure construction and the size of every intermediate automaton (for example `email & !contains(/spam/) & length(8, 30)` gives a 649-state automaton accepting about 3.6 x 10^45 strings).
- **Validator comparison.** Two versions of a validator are compared and classified (equal, stricter, looser, overlapping or disjoint) with shortest witness strings.
- **Automatic test-suite synthesis.** For any validator the system generates positive cases that cover every automaton transition (100% coverage) and four kinds of negative cases, and exports a ready-to-run pytest file. In a demonstration the 123-case suite for a new validator detected the difference from the old one.
- **Exact counting and fair sampling.** The number of strings of each length is computed by dynamic programming over the DFA, finiteness is decided, and strings are sampled exactly uniformly.
- **Constraint pressure.** A per-token measure of how much the specification overrode the model, in bits (average 16.9 bits per output in the benchmark).
- **Verifiable certificates.** Each generation carries the automaton's run on the output and a SHA-256 fingerprint of the automaton; a verifier rebuilds the automaton from the constraint text and replays the run without the model.
- **JSON with a stack.** A deterministic pushdown automaton constrains JSON; a mini JSON schema is compiled into an LL(1) grammar so that keys, order and value types are enforced. The stack is shown live.
- **Efficient token index.** A vocabulary trie walked together with the DFA (up to 114 times faster than testing every token for restrictive constraints) and a lazy index for large automata (a 1,265-state policy: 18.6 s eagerly, 1.3 s for a whole generation lazily).
- **Robust service.** Input and automaton size limits, a time limit, cancellation when the browser disconnects, one consistent error format with the character position of syntax errors, and a refusal (in 0.03 s) of patterns that would need about two million DFA states.
- **Teaching tools.** Step-by-step players for subset construction and table filling that replay the real algorithms.
- **Objective evaluation.** 288 runs, three methods, Wilson confidence intervals, an independent oracle and a raw-output audit page.
- **Quality.** 401 Python tests (with property-based testing against Python's `re` and `json`), 34 frontend tests, strict static typing of the theory core, continuous integration and full documentation.

8. Project Modules

The project is divided into eight modules. Line counts are non-blank source lines (tests excluded).

Module	Name	Main files	Lines	Owner
M1	Regular-expression front end and automata construction	core/regex_parser.py, nfa.py, dfa.py (construction part), dot.py	928	Mankirat
M2	Closure operations, constraint algebra and DFA-to-regex	core/dfa.py (operations), algebra.py, elimination.py	492	Mankirat
M3	Language analysis and validator testing	core/counting.py, testgen.py	266	Lakshit
M4	Context-free side: grammars, LL(1), pushdown automaton, schema	core/cfg.py, pda.py, schema.py	351	Lakshit
M5	Language-model decoding engine and certificates	decoding/*.py, core/certificate.py	831	Mankirat
M6	Backend service (FastAPI)	api/ (18 files)	1,234	Mankirat
M7	Web frontend (React and TypeScript)	frontend/src (21 files)	2,858	Lakshit
M8	Evaluation, command line, quality and packaging	eval/*.py, cli.py, tests, CI, Docker, docs	491 + 2,314 test lines	Lakshit

Total: 7,451 non-test source lines, 2,314 lines of tests, and about 1,300 lines of documentation, in 12 Git commits.

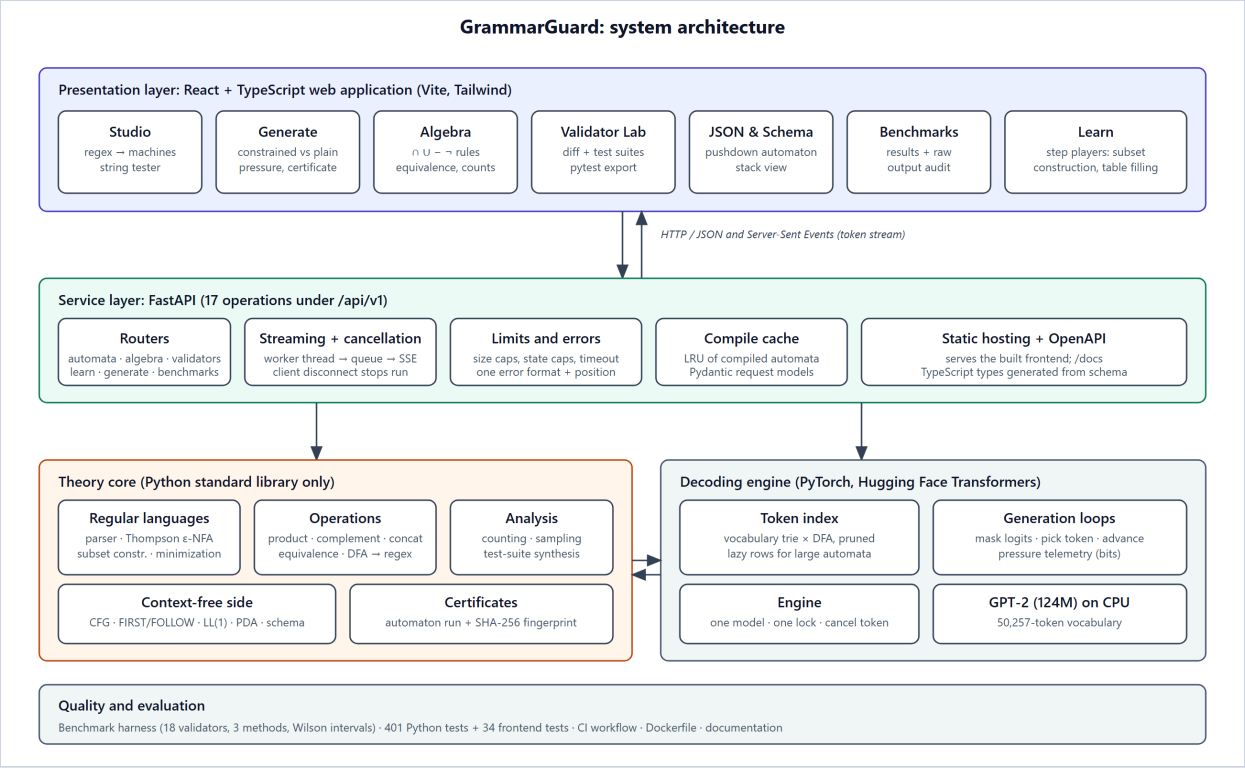


Figure 1: System architecture: presentation, service, theory core, decoding engine and evaluation layers.

9. Module Description

9.1 M1: Regular-expression front end and automata construction

Purpose. Turn a regular expression into the smallest deterministic automaton that accepts the same language, and answer membership queries, using only algorithms from the course.

Regex parser (regex_parser.py)

A tokenizer and a recursive-descent parser follow the LL(1) grammar `union -> concat ('|' concat)*`, `concat -> repeat*`, `repeat -> atom (* | + | ? | {n,m})*`, `atom -> CHAR | CLASS | '(' union ')'`. Precedence (union lowest, then concatenation, then the postfix operators) is encoded in the call chain. All syntactic sugar is rewritten while parsing, so the later stages only ever see union, concatenation and star:

Written	Rewritten as
`R+`	`R R*`
`R?`	`R | epsilon`
`R{n}`	n copies of `R`
`R{n,m}`	`R{n}` followed by (m - n) copies of `R?`
`R{n,}`	`R{n}` followed by `R*`
`[a-z0-9]`	one leaf holding the finite symbol set

Syntax errors raise an exception that carries the character position, which the interface turns into a caret under the faulty character (see Section 13).

Thompson's construction and epsilon-closure (nfa.py)

The AST is converted to an epsilon-NFA by structural induction [7]: every fragment has exactly one start and one accepting state, and the accepting state has no outgoing edges, which makes the glue rules safe. An expression with m AST nodes produces at most 2m states; `(a|b)*abb` produces 14. The epsilon-closure E(S) is computed as graph reachability over epsilon edges only, and the NFA can also be simulated directly by tracking the set of current states.

Subset construction (dfa.py)

Each DFA state is the epsilon-closure of a set of NFA states: q0' = E({q0}), delta'(S, a) = E(move(S, a)), and S is accepting iff it contains an NFA accepting state [8]. Only subsets reachable from q0' are built. The worst case is exponential (`(a|b)*a(a|b){k}` needs 2^(k+1) states because the DFA must remember the last k + 1 characters), so the construction accepts a state cap and raises a dedicated error instead of exhausting memory.

The symbol OTHER. A language model can emit any character, not only those in the regex. All characters outside the alphabet behave identically, so every DFA has one extra symbol, OTHER, with an edge from every state. For an ordinary regex all OTHER edges go to the dead state. The symbol is explicit rather than implicit because complement needs it: for the language *not a*, a character outside the alphabet must lead to an accepting state.

Minimization

Two algorithms compute the same Myhill-Nerode classes [10, 11]. **Table filling** marks every pair of states distinguished by the empty string (one accepting, one not) and then repeatedly marks a pair if some symbol

sends it to a marked pair; unmarked pairs are merged (O(n^2 |Sigma|) per round). It is used for small automata and for the step-by-step player, which records each round. **Moore's partition refinement** [12] starts from the partition {F, Q - F} and splits blocks until stable (O(n |Sigma|) per round); it is used above 64 states, where it turned the minimization of a 1,300-state policy from seconds into 0.4 s. A test checks that both produce identical tables. After minimization the states are renumbered in breadth-first order, so the minimal DFA has a canonical table whose SHA-256 hash is its signature; equivalent regexes therefore share cache entries and certificate fingerprints.

Live and dead states

A state is live if an accepting state is reachable from it (backward breadth-first search from the accepting states). A string that reaches a non-live state can never be completed to a valid string. This is the notion on which constrained decoding rests.

Verification. The NFA, the DFA and the minimal DFA are compared with Python's `re.fullmatch` on random strings (Hypothesis) for 16 realistic patterns and 300 random regexes; minimization is checked for idempotence and equivalence with the unminimized DFA. Example: `(a|b)*abb` gives 14 NFA states, 6 DFA states and 5 minimal states (four visible states plus the dead state).

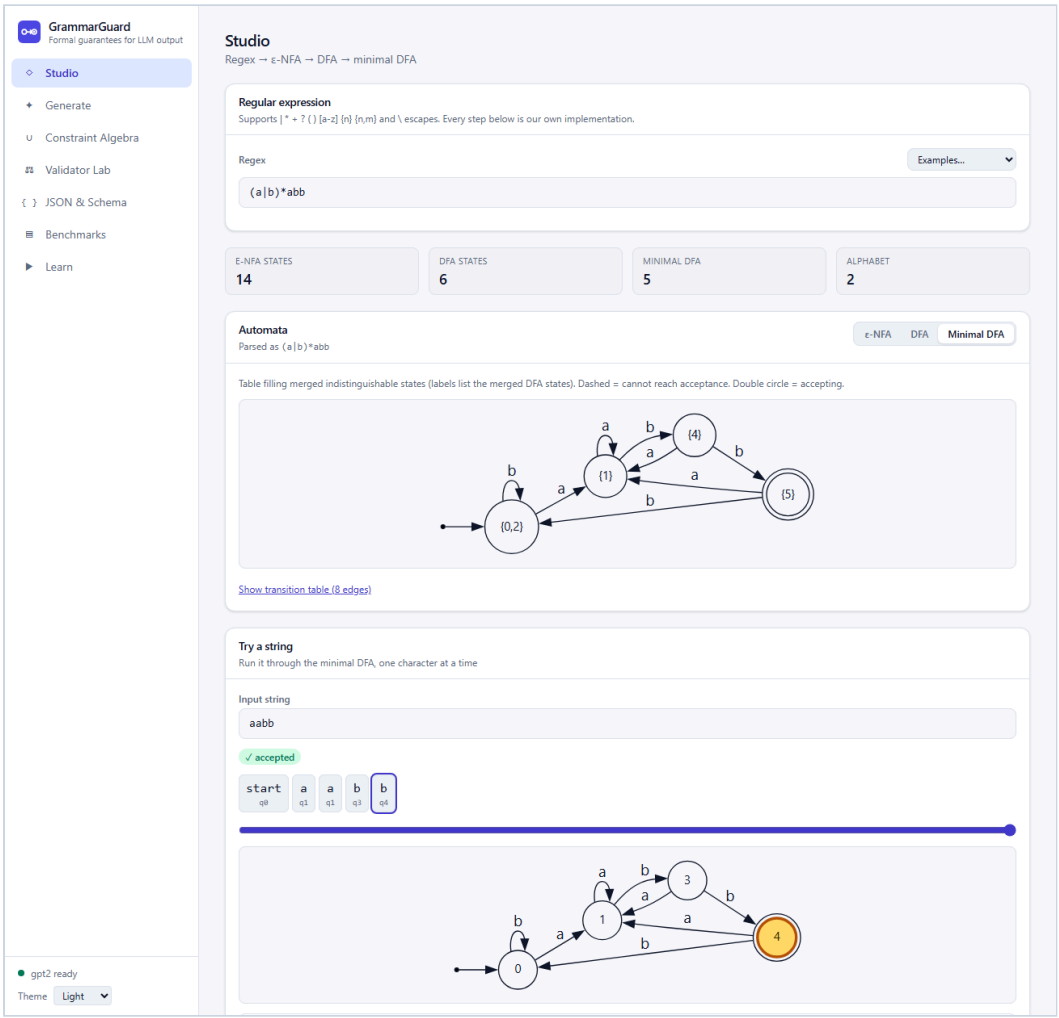


Figure 2: Studio page: the three state counts (14, 6, 5), the minimal DFA, and the string tester with the path of `aabb` through the automaton.

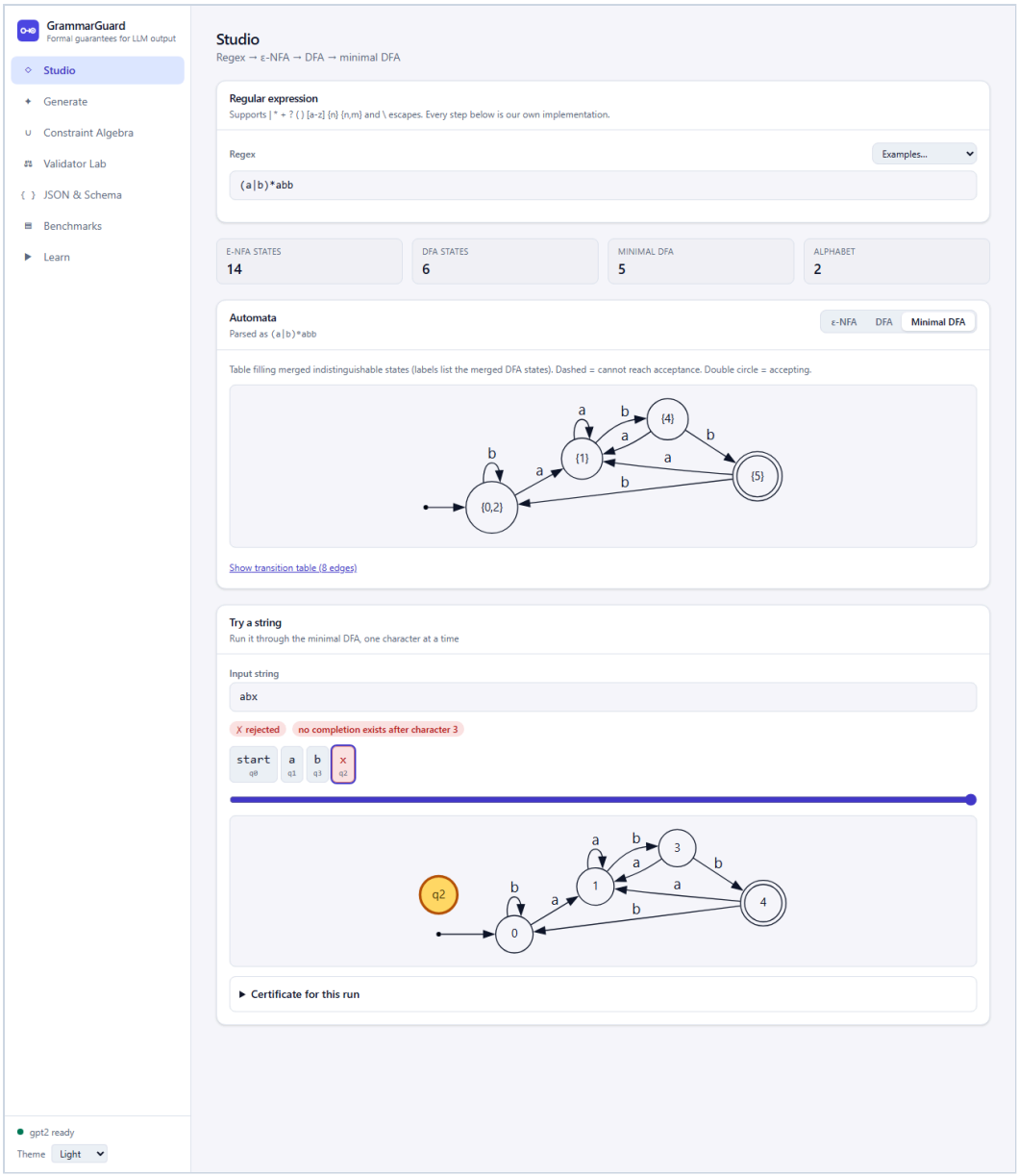


Figure 3: The string abx falls into the dead state (the isolated yellow circle q2): no completion exists after the third character.

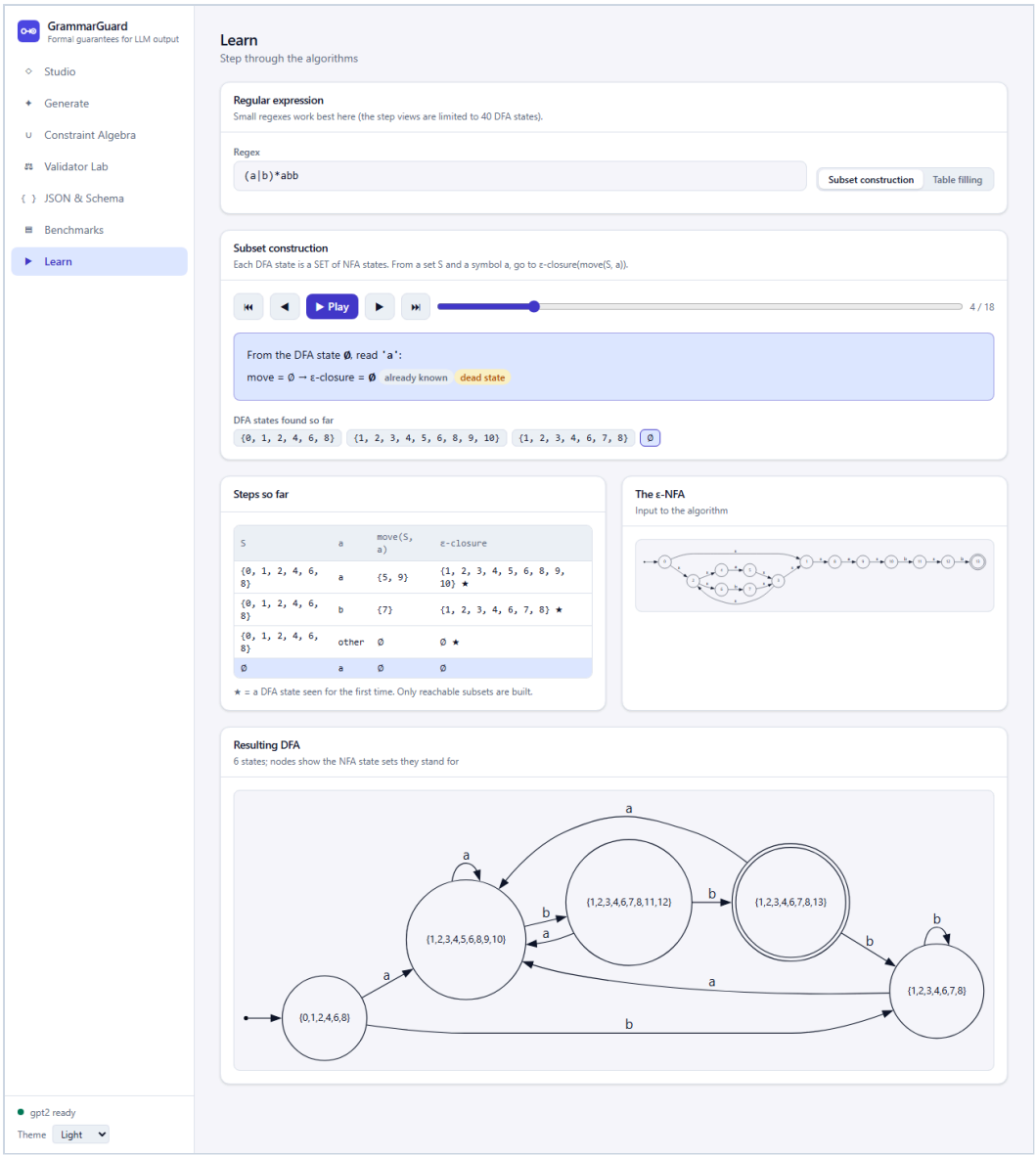


Figure 4: Learn page, subset construction: one (set, symbol) step at a time, produced by the real algorithm.

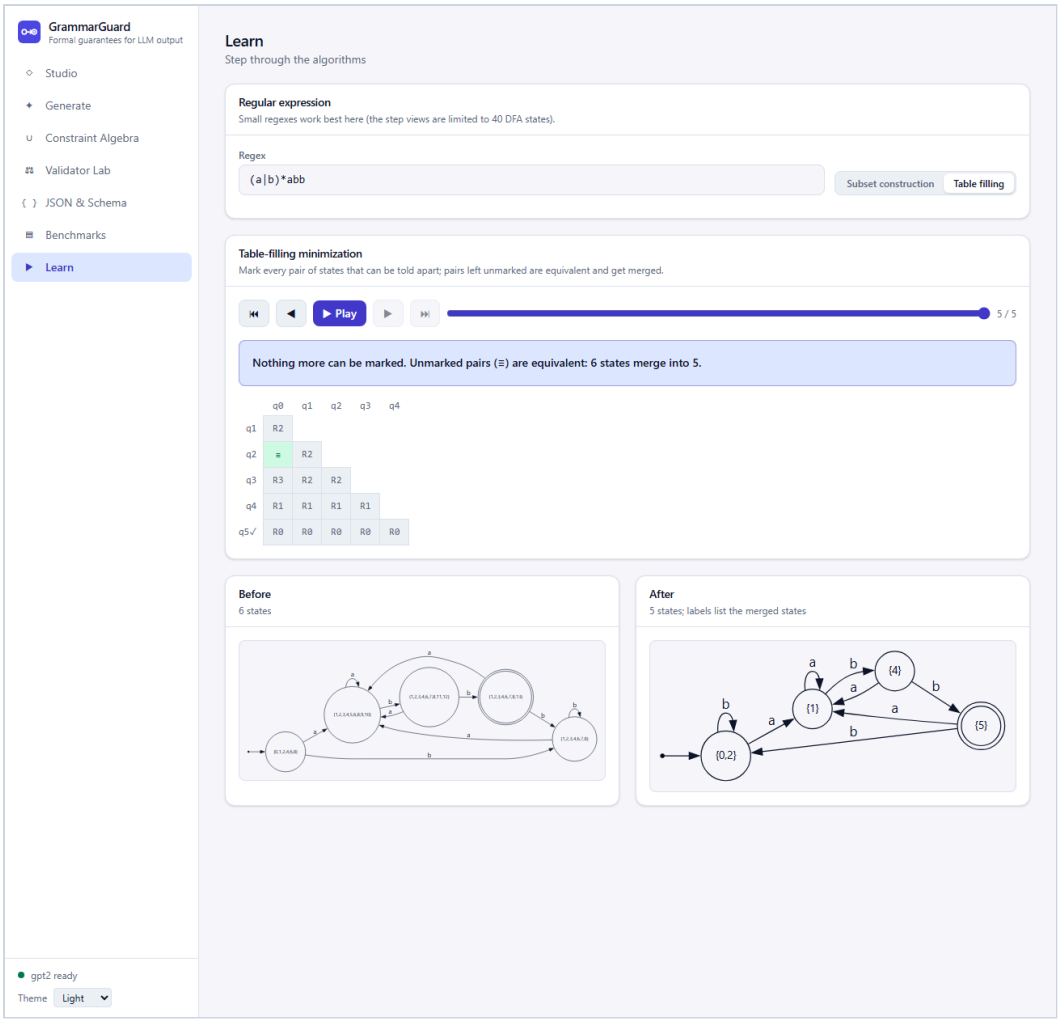


Figure 5: Learn page, table filling after the last round: 14 of 15 pairs marked, one equivalent pair (shown as the green mark), so 6 states merge into 5.

9.2 M2: Closure operations, constraint algebra and DFA-to-regex

Purpose. Use the closure properties of regular languages as practical operations, decide equivalence, and convert automata back to regular expressions.

Operation	Construction (`dfa.py`)
Intersection, union, difference, symmetric difference	Product DFA: states (p, q), delta((p,q),a) = (delta1(p,a), delta2(q,a)); the accepting rule depends on the operation. Alphabets are unified; a symbol missing from one side follows that side's OTHER edge.
Complement	Swap accepting and non-accepting states; valid because the DFA is complete, OTHER included.
Concatenation, star	Glue NFAs with epsilon edges (accepting states of A to the start of B; a new accepting start for star), then determinize and minimize.
Equivalence	L1 = L2 iff L1 symmetric-difference L2 is empty. Build that product and run breadth-first search to the nearest accepting state; the string found is a *shortest counterexample*.

Constraint algebra (`algebra.py`). A recursive-descent parser for expressions such as `email & !contains(/spam/) & length(8, 30)` (grammar: `expr -> term ('|' term)*`, `term -> unary (('&' | '-') unary)*`, `unary -> '!' unary | primary`) evaluates each operator with the construction above and

minimizes after every step. Regexes are written between slashes; 20 built-in names (`digits`, `email`, `iso_date`, `time24`, `us_phone`, `uuid`, ...) and user-defined names are supported. `contains(A)` is `ANY* A ANY*` and `length(n,m)` is a counter automaton. A state cap protects against runaway products, and errors carry the position in the expression.

DFA to regex (`elimination.py`). State elimination (Kleene's theorem [9]) adds a new start and final state and removes states one by one, replacing p -> q -> r by `R(p,r) | R(p,q) R(q,q)* R(q,r)`. Simplifying constructors keep the result readable (single-symbol alternatives merge into a class, `R R*` becomes `R+`, repeated factors become `{n}`). The expression can grow exponentially, so the service attempts it only for automata of at most 40 states and the algorithm itself has a size budget.

Examples measured. `digits & !/0[0-9]*/ & length(2, 4)` gives a 6-state automaton accepting 9,990 strings, with the equivalent regex `[1-9][0-9]([0-9][0-9]?)?`. The policy `email & !contains(/spam/) & length(8, 30)` is built in seven steps (12, 6, 5, 5, 33, 32 and finally 649 states). `(a|b)*` and `(a*b*)*` are reported equivalent, while `(a|b)*` and `(a|b)*a` differ with the shortest counterexample being the empty string.

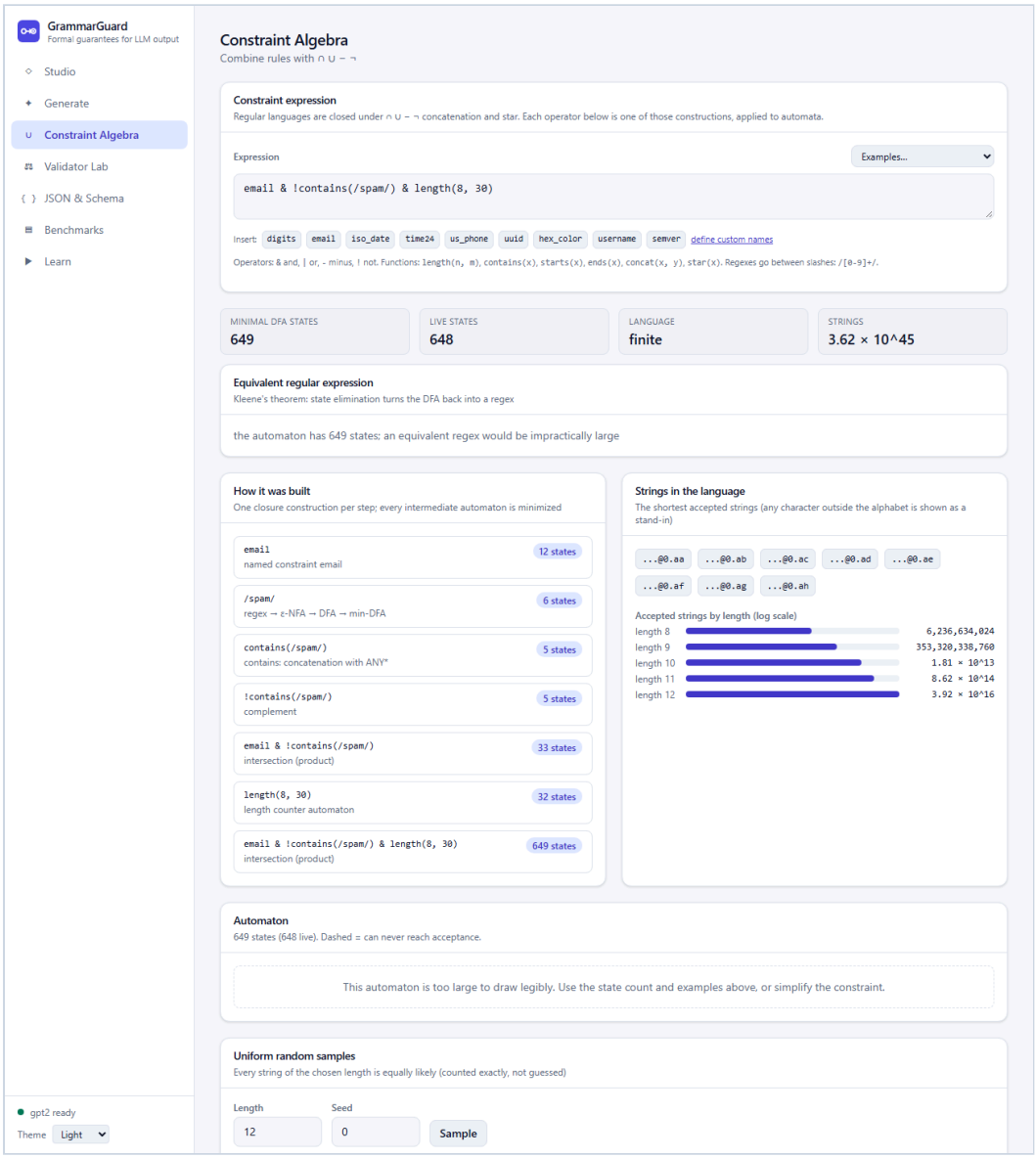


Figure 6: Constraint Algebra page: the policy automaton (649 states), the construction steps, the shortest accepted strings and the count of valid strings per length.

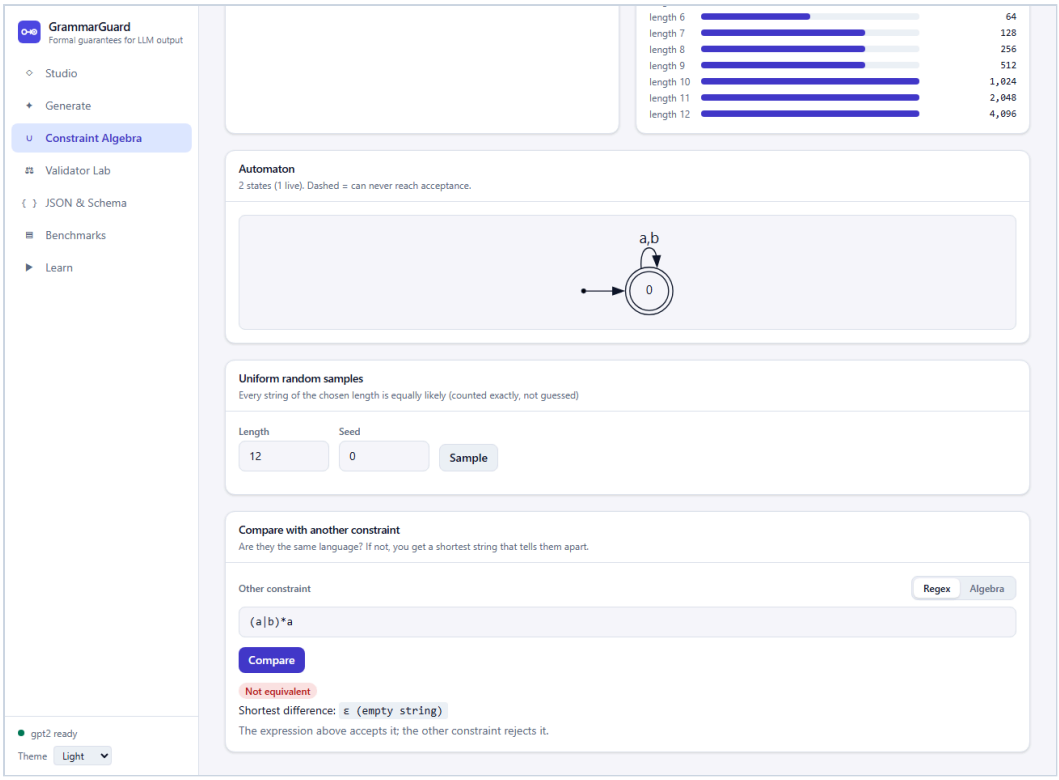


Figure 7: Equivalence check: `(a|b)*` versus `(a|b)*a`; the shortest difference is the empty string.

9.3 M3: Language analysis and validator testing

Purpose. Answer quantitative questions about a regular language and turn a validator into a test suite.

- **Counting.** N(q, 0) = 1 if q is accepting, N(q, k) = sum over symbols a of N(delta(q, a), k - 1). This counts the accepted strings of each length in O(k n |Sigma|) time although the counts are exponentially large (counts are exchanged as decimal strings because JavaScript numbers lose precision above 2^53).
- **Finiteness and size.** A language is infinite iff some useful state (reachable and live) lies on a cycle, found by depth-first search with white/grey/black colouring; a finite language has no string longer than n - 1.
- **Exact uniform sampling.** From state q with k symbols left, symbol a is chosen with probability N(delta(q,a), k-1) / N(q, k); the probability of any accepted string is the telescoping product 1 / N(start, k), so every string is equally likely (checked statistically in the tests).
- **Shortest strings.** The k shortest accepted strings in shortlex order. A first version searched all viable prefixes and exhausted memory on dense languages (Section 13); the final version uses a feasibility table can[r] (states from which acceptance is possible in exactly r more symbols) and a depth-first search that follows only guaranteed branches.
- **Test-suite synthesis (`testgen.py`).** Positive cases: for every useful transition p -a-> r, the string access(p) + a + completion(r), which gives 100% transition coverage. Negative cases of four kinds: *boundary* (valid prefix plus the first breaking character), *substitution* (a valid string with one character replaced by a likely mistake such as a space or underscore), *deletion* (a required character left out) and *truncation* (stopping too early). Substitution and deletion were added after a mutation test showed that boundary cases alone miss validators that over-accept a separator or a length. The suite can be exported as a runnable pytest file.

Demonstration. For the validator `[0-9]{3}[-_][0-9]{2}` the system generates 123 cases (47 positive, 76 negative) with 100% transition coverage. The downloaded pytest file passed all 123 cases against this validator; run against the *old* validator `[0-9]{3}-[0-9]{2}` it failed on exactly one case, `000_00`, showing that the suite detects the change.

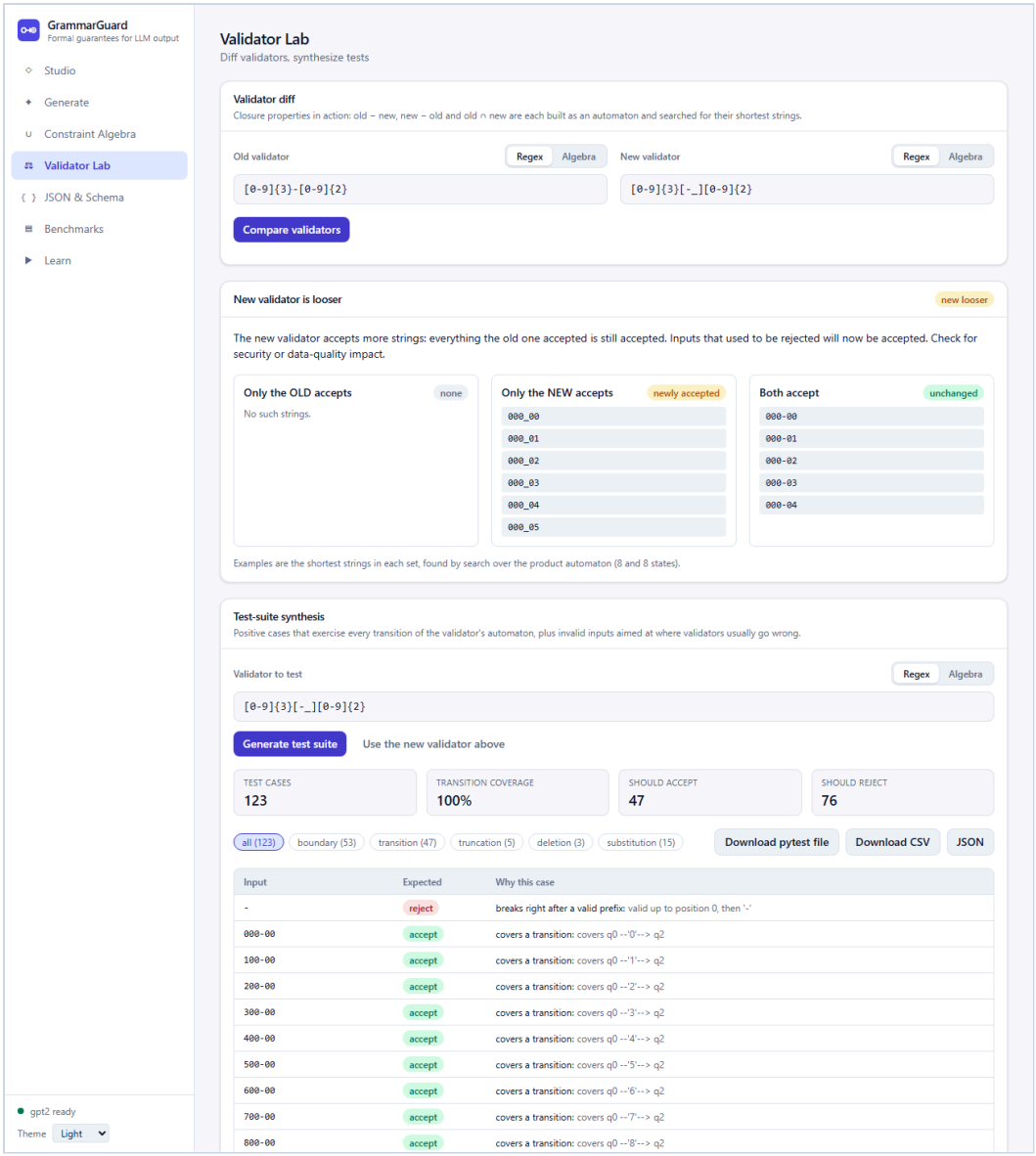


Figure 8: Validator Lab: the new validator is classified as looser than the old one (witness `000_00`), followed by the generated test suite with its coverage.

9.4 M4: Context-free side: grammars, LL(1), pushdown automaton and schema compiler

Purpose. Constrain nested structures that no finite automaton can enforce. Balanced brackets cannot be recognised with finite memory (a consequence of the pumping lemma [16, 17]), so JSON needs a stack.

- **Grammars (`cfg.py`).** Terminals are single characters or character classes, so the same machinery can consume text character by character. Nullable symbols, FIRST and FOLLOW are computed by fixed-point iteration. The LL(1) table is M[A, c] = alpha if c is in FIRST(alpha), or alpha is nullable and c is in FOLLOW(A) [14, 18]. A cell with two productions raises an error that lists the conflicts.

- **JSON grammar.** A character-level LL(1) grammar for a JSON subset: objects, arrays, lowercase strings, non-negative integers without leading zeros, `true`, `false` and `null`, written compactly without whitespace.
- **Pushdown automaton (`pda.py`).** One state, acceptance by empty stack. If the top is a nonterminal A, pop it and push the right-hand side of M[A, c] (the *expand* move, chosen by the next character); if the top is a terminal containing c, pop it (*match*). At the end of input only epsilon expansions (look-ahead EOF) may remain. Every table cell has at most one entry, so the automaton is deterministic [15]. Stacks are immutable tuples so a candidate token can be tried without disturbing the real stack.
- **Schema compiler (`schema.py`).** A mini JSON schema (object with fixed key order, array with `minItems` 0 or 1, string, integer, boolean, null, enum) is compiled by structural recursion into an LL(1) grammar with one nonterminal per schema node; enums become a trie so common prefixes remain LL(1). Building the array rule exposed a genuine LL(1) conflict, which was fixed by left-factoring (Section 13).

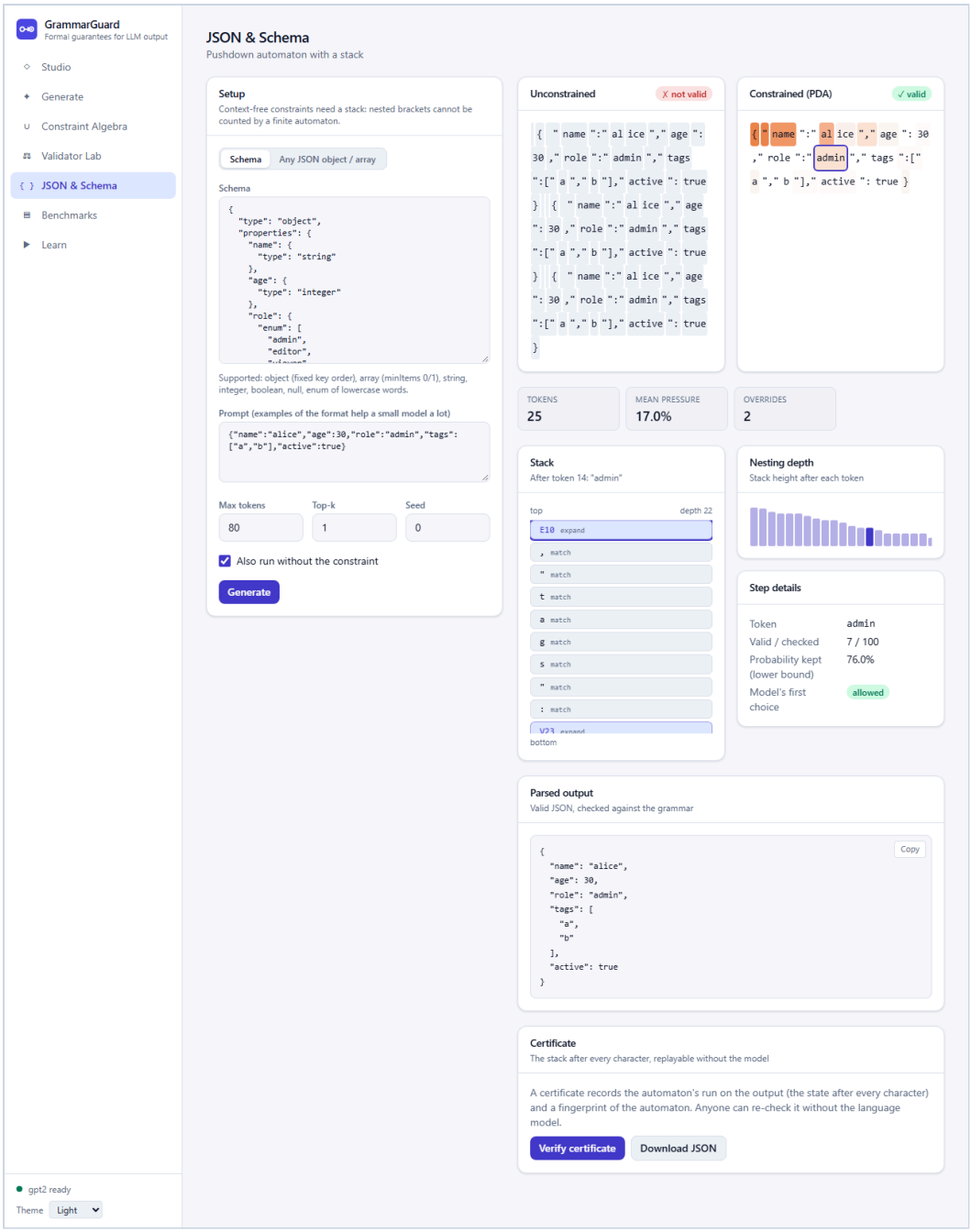


Figure 9: JSON and Schema page: the constrained output is valid and conforms to the schema; the stack after the token `admin` *shows what the automaton still has to match (depth 22).*

9.5 M5: Language-model decoding engine and certificates

Purpose. Connect the automata to GPT-2 so that generation can only follow valid paths, and make the result checkable.

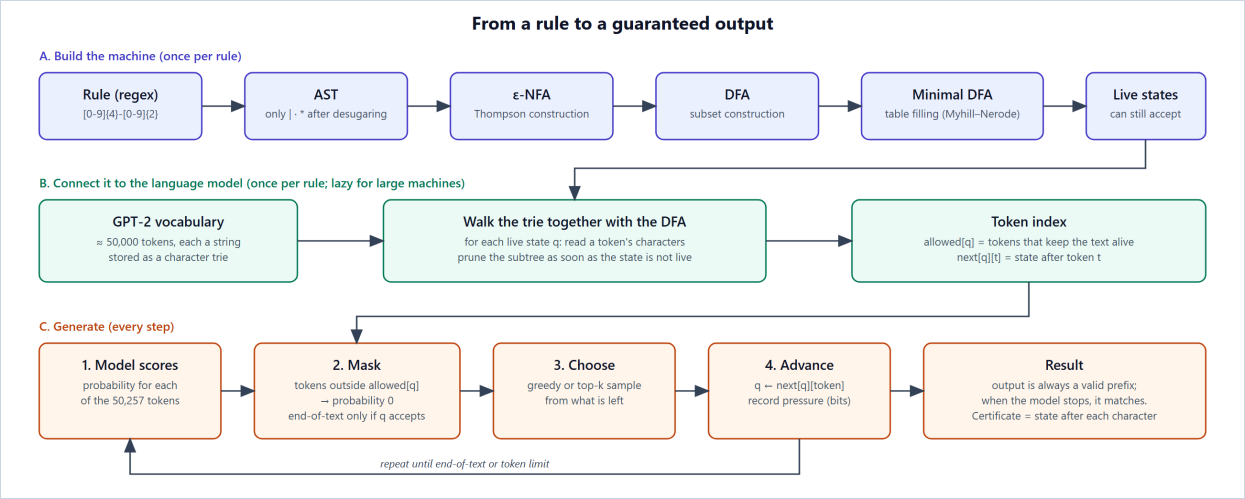


Figure 10: From a rule to a guaranteed output: building the machine, building the token index, and the generation loop.

Token index (token_index.py)

A model emits multi-character tokens while a DFA reads characters. For every live state q and token t the DFA is run over the characters of t; t is *allowed* in q iff the end state is live, and the end state is recorded as next[q][t]. GPT-2's byte-level vocabulary [21] is decoded to exact strings by inverting its byte-to-unicode map; tokens that are fragments of a multi-byte character or special tokens are excluded.

Guarantee. If every emitted token keeps the state live and end-of-text is allowed only in accepting states, then the output is always a prefix of some string of the language, and if generation ends with end-of-text it is a string of the language (induction over the tokens). The model cannot get stuck, because GPT-2's vocabulary contains a token for every ASCII character, so a live state always has an allowed token.

Vocabulary trie. Instead of running the DFA separately over each of about 50,000 tokens, a character trie of the vocabulary (97,317 nodes) is walked together with the DFA; when the DFA reaches a non-live state the whole subtree is pruned, which is valid because non-live states can only lead to non-live states. The index is identical to the naive one (tested on random vocabularies and the real one). Measured speed-up depends on how restrictive the constraint is:

Constraint	Live states	Per-token loop	Trie	Speed-up
ISO date	11	0.48 s	0.00 s	114x
Integer	4	0.15 s	0.00 s	37x
E-mail	10	0.48 s	0.08 s	5.8x
Username	17	0.70 s	0.18 s	3.8x
`!contains(/spam/)`	4	0.17 s	0.29 s	0.6x (slower)
E-mail policy	1,265	50 s	18.6 s	2.7x

The one slower case is instructive: when almost every token is allowed nothing is pruned and the trie's bookkeeping costs more. For large automata a **lazy index** computes the allowed set of a state the first time generation reaches it, because a generation visits only a few dozen states; for the 1,265-state policy a complete 20-token generation took 1.3 s instead of an 18.6 s index build.

Generation loop and constraint pressure (generate.py)

The loop is written by hand (not the library's `generate`). At each step the logits of tokens outside allowed[q] are set to minus infinity, the end-of-text logit is kept only if q is accepting, a token is chosen greedily or by top-k sampling, and the state advances. With p the model's distribution and A the allowed set, each step records mass_kept = p(A), pressure = 1 - p(A), and **cost_bits = -log2 p(A)**. This is exactly the Kullback-Leibler divergence [24] of q = p(. | A) from p, because KL(q||p) = sum over A of (p_t/Z) log((p_t/Z)/p_t) = -log Z with Z = p(A); it is the information the constraint adds. It is computed in log space so tiny masses do not underflow, and it also records whether the model's own first choice was allowed (an *override* otherwise).

PDA decoding (pda_generate.py)

A PDA configuration includes an unbounded stack, so no finite table can be precomputed. Each step checks the model's 100 most likely tokens against the current stack and walks the whole vocabulary trie (carrying the stack) only when none of them is valid. Greedy decoding over the top 100 equals greedy decoding over the full vocabulary, because any valid token inside the top 100 outranks every token outside it; with sampling the cut is an approximation.

Engine and certificates (engine.py, certificate.py)

The engine owns the model, a lock (CPU GPT-2 must not be driven from several threads), an LRU cache of token indexes and a cancellation token polled between tokens, so a client that disconnects stops its generation after at most one more token. A **certificate** stores the automaton's run on the output (the state, or the stack for the PDA, after every character), the token boundaries, the constraint text and a SHA-256 fingerprint of the automaton. The verifier rebuilds the automaton from the constraint text, compares the fingerprint and replays the run one transition at a time in linear time, trusting neither the generator nor the model. *Valid* (the certificate is truthful) is reported separately from *accepted* (the text is in the language); a certificate can honestly prove that a text is not accepted.

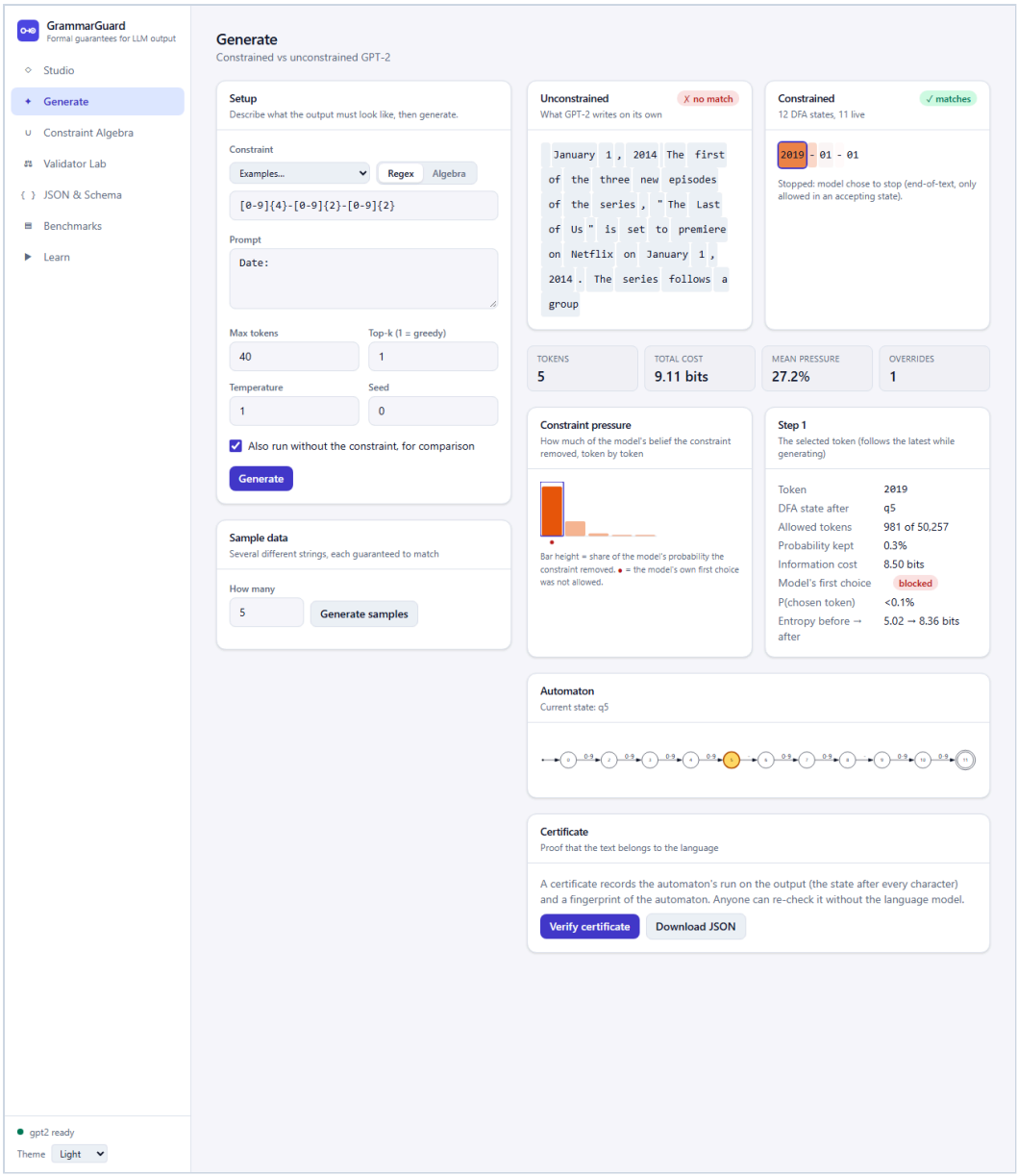


Figure 11: Generate page: plain GPT-2 writes a sentence (no match) while the constrained output `2019-01-01` matches; the step panel shows that the model's own first choice was blocked (8.50 bits).

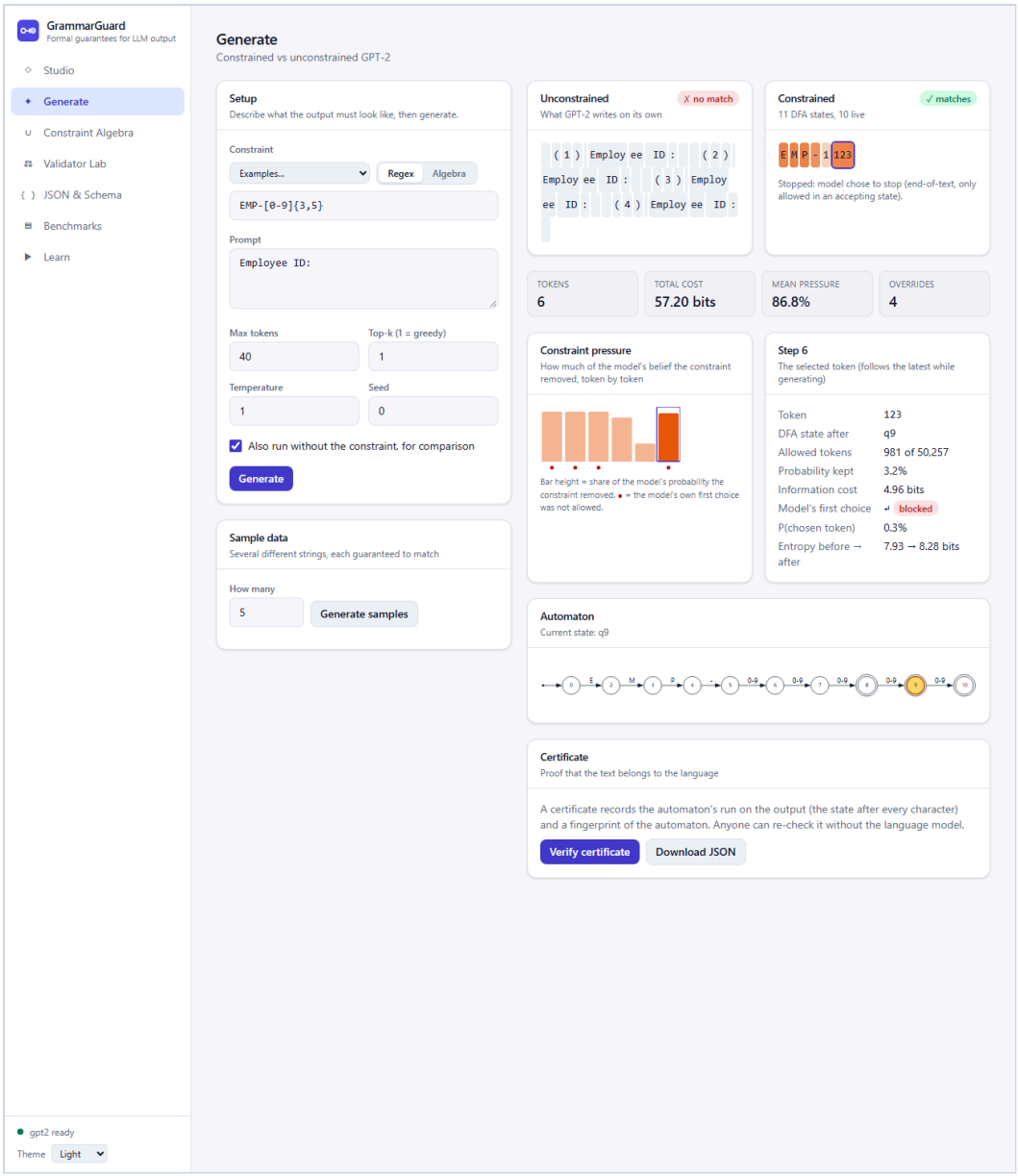


Figure 12: When the model fights the format: for an employee ID the constraint costs 57.2 bits at 86.8% pressure with four overrides.

9.6 M6: Backend service (FastAPI)

Purpose. Expose all functionality as a documented, safe HTTP API.

- **17 operations** under `/api/v1` (listed in Section 11), with interactive documentation generated from the code at `/docs`.
- **Streaming.** Generation endpoints answer with Server-Sent Events. The model is blocking CPU code, so each request runs in its own worker thread that feeds a queue; if the client disconnects the response is cancelled, a cancel token is set, and the worker stops at the next token and releases the model lock. A timer stops runs longer than the time limit and reports `timeout`.
- **Limits protect a shared server.** Regex length (500), expression length (2,000), input text (10,000), prompt (4,000), minimal-DFA states (2,000), unminimized subset-construction states (5,000), tokens per generation (200), batch size (50) and a generation time limit (120 s). All are configurable through environment variables.

- **One error format.** Every failure is `{error: {code, message, position, details}}` with codes such as `regex_syntax`, `algebra`, `schema`, `limit_exceeded`, `not_expressible` and `validation`; mistakes by the user are HTTP 422 and only genuine bugs are 500.
- **Compile cache and logging.** An LRU cache of compiled automata, a request identifier on every response and one JSON log line per request.
- **Static hosting.** The built frontend is served by the same process, so production needs one process and one port.

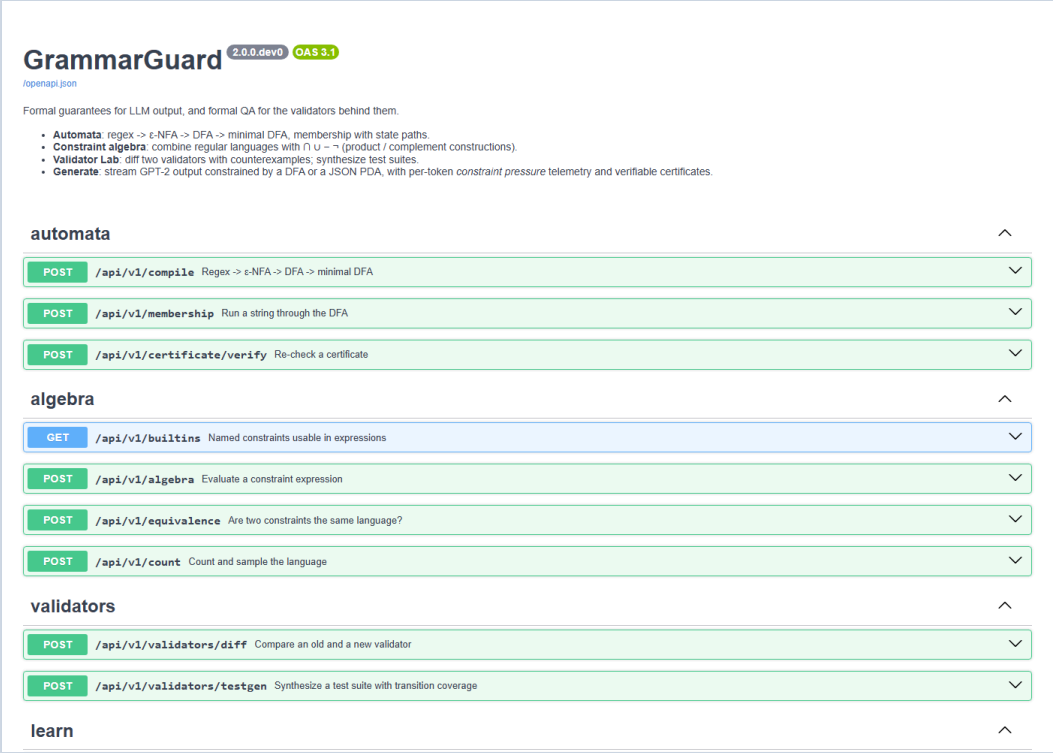


Figure 13: Automatically generated interactive API documentation.

9.7 M7: Web frontend (React and TypeScript)

Purpose. Make every capability usable and visible. The application has seven pages (see Figure 1):

Page	What it does
Studio	Compile a regex; view the epsilon-NFA, DFA and minimal DFA with state counts and the transition table; run strings through the DFA step by step.
Generate	Compare plain and constrained GPT-2 side by side, streaming, with per-token pressure, the current automaton state, a certificate and sample data.
Constraint Algebra	Evaluate combined rules; view the construction steps, the equivalent regex, counts and uniform samples; compare two constraints.
Validator Lab	Compare an old and a new validator with witnesses; generate and download test suites.
JSON & Schema	Constrain JSON, optionally by schema, and watch the pushdown automaton's stack and depth.
Benchmarks	Measured results with confidence intervals, per-category and per-validator tables, and an audit of every raw output.
Learn	Play or step through subset construction and table filling on any small regex.

- **Typed against the backend.** TypeScript types are generated from the backend's OpenAPI schema, so a renamed field breaks the frontend build instead of production; the token-stream events are typed by hand.
- **Streaming.** Because the browser's `EventSource` cannot send POST requests, a small stream reader with its own Server-Sent-Events parser (handling CRLF line ends, keep-alive comments and events split across chunks) is used; aborting the request cancels the generation on the server.
- **Drawing.** The backend sends Graphviz DOT text and the browser draws it with Graphviz compiled to WebAssembly, loaded only when the first graph is shown; the current state is highlighted by adding one node statement to the DOT text.
- **Presentation.** Light and dark themes, a layout that collapses to a top bar on narrow screens, semantic roles and labels on charts and controls, and server errors shown with their code and a caret.

9.8 M8: Evaluation, command line, quality and packaging

Benchmark design (`eval/`). Eighteen validators in six categories (dates, phone numbers, identifiers, e-mail and usernames, numbers, and combined policies), each with two prompts: a bare prompt, and a few-shot prompt that shows examples of the format (the best a user can do without a constraint). Each prompt is run greedily and with top-20 sampling (two seeds) by three methods: **plain GPT-2**, **best-of-five rejection sampling** (sample up to five times and keep the first valid output, the usual workaround) and **GrammarGuard**: 288 runs in all. Validity means the *whole* output is accepted, as a real validator requires; every regex output is also judged by Python's `re.fullmatch` (224 outputs, no disagreement). Each baseline is additionally reported with *prefix extraction*, which counts a run if any non-empty prefix of the output matches, the most a post-processing step could rescue. Intervals are 95% Wilson intervals [23].

A correction made during evaluation. The first benchmark run used few-shot prompts that ended in the middle of an item (for example `EMP-`), which made the baseline fail because of the prompt rather than the model, and a retry baseline that received no credit for valid prefixes. Both were found by reading the raw outputs, fixed, and the whole benchmark was re-run; a test now requires every prompt to end at an item boundary. The results below are from the corrected run.

Method	Output satisfies the validator	Mean tokens	Mean seconds
Plain GPT-2, as is	0% (0 of 108; 95% interval 0-3%)	29.3	0.85
Plain GPT-2, extracting a valid prefix	11% (6-18%)		
Best-of-5 retries, as is	3% (2 of 72; 1-10%)	144.0	4.21
Best-of-5 retries, extracting a valid prefix	17% (10-27%)		
GrammarGuard	**100% (108 of 108; 97-100%)**	7.0	0.26

Constrained decoding cost on average 16.9 bits per output (mean pressure 48%, 2.8 overrides) and took 1.34 times as long per token as unconstrained decoding (excluding the one-off index build, which is milliseconds for ordinary validators). Plain GPT-2 almost never stops where the format ends (104 of 108 runs ran to the token limit), so per-token cost, not total time, is the fair comparison. Three constrained runs stopped at the token limit rather than at end-of-text; all were the open-ended `decimal` validator, where the text is a genuinely valid number but repetitive. GPT-2 is a small model; a larger model would do better unaided, so the gap should be read as showing that GPT-2 cannot be trusted with these formats.

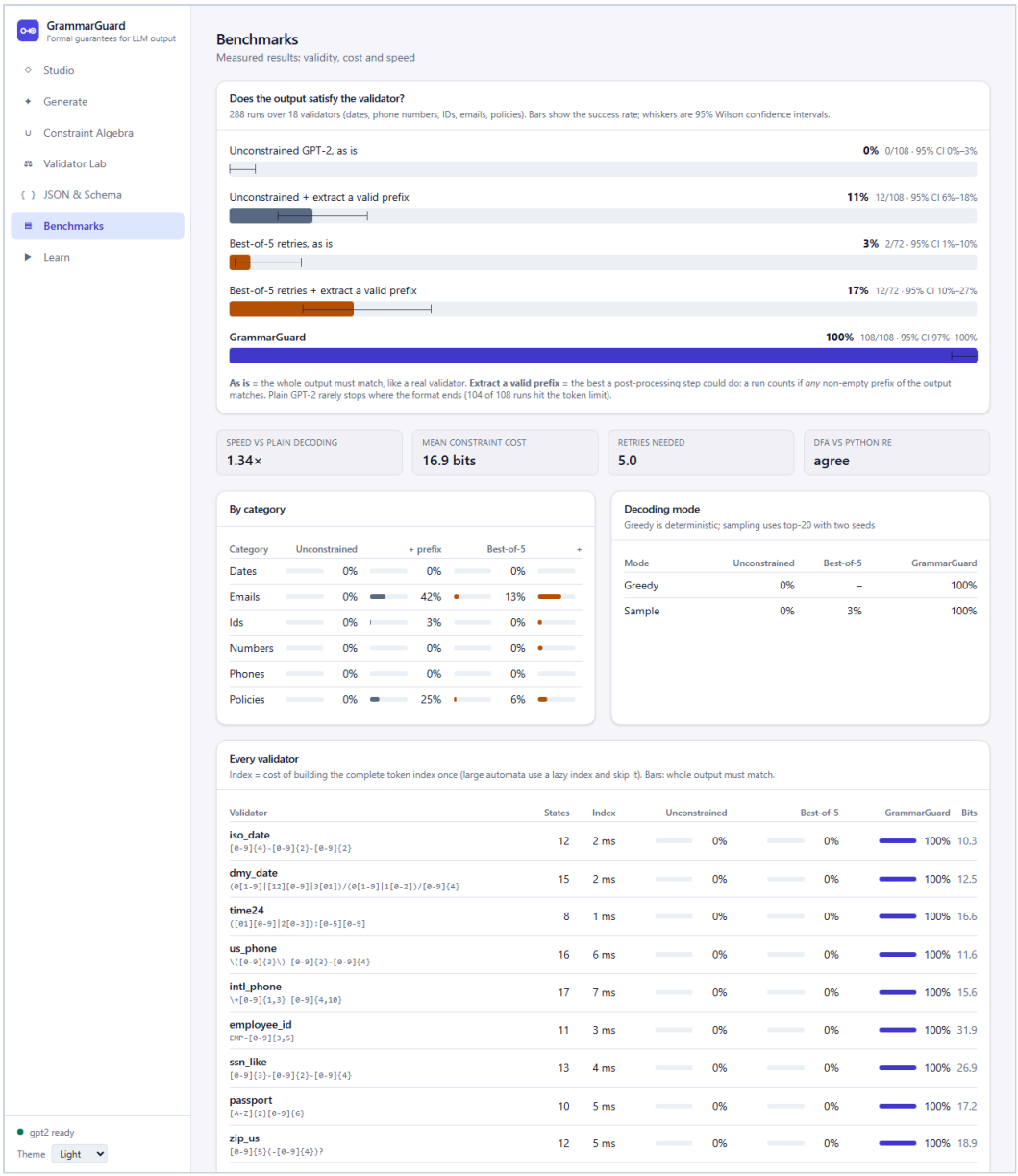


Figure 14: Benchmarks page: five bars with 95% confidence-interval whiskers, summary cards, and per-category and per-validator tables.

Command line, tests and packaging. `python -m grammarguard.cli` compares plain and constrained generation for a regex. The 401 Python tests include property-based comparisons with `re` and `json`, brute-force checks of every algebraic construction, a fake language-model engine that makes the streaming, queueing, timeout and cancellation paths fast and deterministic, and tests that drive the real GPT-2; the 34 frontend tests exercise every page against a mocked backend. A continuous-integration workflow, a multi-stage Dockerfile (frontend build, CPU-only PyTorch, model baked in, offline, unprivileged user) and documentation (`README`, theory notes, API reference, demo guide and viva notes) complete the delivery.

10. Role of Team Members

The work is divided by module so that each member owns a complete vertical slice, a mix of theory-heavy and engineering work, and a comparable amount of code (47% / 53% of the non-test source lines).

	Mankirat Singh Bhatia (24BIT0048)	Lakshit Bhatia (24BIT0012)
Modules	M1 Regex front end and automata construction; M2 Closure operations, algebra and DFA-to-regex; M5 Decoding engine and certificates; M6 Backend service	M3 Language analysis and validator testing; M4 Context-free side (CFG, LL(1), PDA, schema); M7 Web frontend; M8 Evaluation, quality and packaging
Source lines owned	3,485 (47%)	3,966 (53%)
Theory topics	Regular expressions, Thompson's construction, epsilon-closure, subset construction, DFA minimization, closure properties, equivalence, Kleene's theorem; the token-to-automaton bridge and the correctness guarantee	Counting and decision procedures (finiteness, emptiness), context-free grammars, FIRST/FOLLOW, LL(1) parsing, pushdown automata, the pumping-lemma argument for JSON; test generation from automata
Engineering	FastAPI service, streaming and cancellation, limits and error handling, token index (trie, lazy), generation loop, telemetry, certificates	React frontend and typed API client, step players, benchmark harness and statistics, test suites, continuous integration, Docker, documentation
Viva focus	Walk through regex -> NFA -> DFA -> minimal DFA; product and complement; equivalence; why the output is guaranteed to match; trie pruning; certificates	Walk through grammars and LL(1) tables; why a stack is needed; counting and uniform sampling; how test suites are generated; how the benchmark was designed and why it is fair

Joint responsibilities. Both members share responsibility for the requirements, for the design decisions recorded in the project's decision log, for the demonstration and for this report; each member is also responsible for the tests of the modules he owns and must be able to explain the whole system, not only his own modules.

11. Software Description

11.1 Overview

GrammarGuard is a web application backed by a Python service. A user states what the text must look like (a regex, a combination of rules, or a JSON schema); the system compiles that specification into automata, and uses them either to analyse the specification or to constrain a language model. The architecture is shown in Figure 1 and the processing pipeline in Figure 10.

11.2 System requirements and installation

Item	Requirement
Operating system	Linux, macOS or Windows (developed on Ubuntu under WSL2)
Python	3.11 or later
Node.js	20 or later (only to build the frontend)
Hardware	A CPU is sufficient (no GPU); about 8 GB of RAM; about 500 MB of disk for GPT-2
Network	Needed once to download GPT-2; none afterwards (the Docker image includes the model)

Listing 1: Installation and start-up commands

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -e ".[ml,api,dev]"
make web-install
make serve            # builds the frontend and serves API + UI on http://localhost:8000
# alternatively:  docker compose up --build
```

11.3 Functional description

The seven pages and their functions are described in Section 9.7 and illustrated in Figure 2 to Figure 14. The main workflows are:

1. **Analyse a rule:** enter a regex in Studio; inspect the epsilon-NFA, DFA and minimal DFA; test strings; replay the algorithms in Learn.
2. **Generate with a guarantee:** enter a rule and a prompt in Generate; compare the plain and constrained outputs; inspect the pressure of each token; verify the certificate.
3. **Combine and compare rules:** build a policy in Constraint Algebra; check equivalence; compare an old and a new validator in Validator Lab; download the generated test suite.
4. **Constrain JSON:** edit a schema in JSON & Schema and watch the stack.
5. **Evaluate:** read the Benchmarks page, or re-run `python -m grammarguard.eval`.

11.4 Application programming interface

Method	Path	Purpose
POST	/compile	Regex to epsilon-NFA, DFA and minimal DFA (drawings, tables, counts)
POST	/membership	Run a string through the DFA; state path, point of failure, certificate
POST	/certificate/verify	Re-check a certificate without the language model

Method	Path	Purpose
GET	/builtins	Named constraints usable in expressions
POST	/algebra	Evaluate a constraint expression; size, examples, equivalent regex, steps
POST	/equivalence	Same language? Otherwise a shortest counterexample
POST	/count	Counts per length, finiteness, exact uniform samples
POST	/validators/diff	Old versus new validator with witness strings
POST	/validators/testgen	Test-suite synthesis (JSON, CSV or pytest)
GET	/learn/subset, /learn/minimize	Step traces of the real algorithms
POST	/generate	Stream DFA-constrained generation (SSE)
POST	/generate/json	Stream PDA-constrained JSON generation (SSE)
POST	/generate/batch	Several distinct valid samples
GET	/benchmarks	Stored evaluation results
GET	/system/health, /system/models	Liveness, model state and server limits

All paths are under `/api/v1`. The list contains 17 operations (the learn and system rows each contain two).

11.5 Quality assurance

Aspect	Approach and result
Correctness of the automata	Differential testing against Python's `re.fullmatch` and `json.loads` on thousands of random strings (Hypothesis), including near-misses of matching strings; every algebraic construction (complement, product, concatenation, star, minimization, DFA-to-regex round trip) checked by brute force; shortest counterexamples verified minimal by exhaustive search.
Python tests	401 tests; the model-free subset runs in about half a minute
Frontend tests	34 tests covering the SSE parser, API client, helpers and every page, including error paths
Static checking	`mypy` on the whole theory core; `ruff` on all Python; strict TypeScript
Real browser	All seven pages were checked in headless Chrome and screenshotted; four layout defects were found and fixed that way
Safety	Size and state caps, time limit, cancellation, input validation before any streaming starts
Continuous integration	GitHub Actions workflow runs the fast Python tests and the frontend type-check, tests and build

11.6 Limitations

- The model is GPT-2: the *format* of the output is guaranteed, its *content* is only as good as a small model.
- The JSON subset is compact (no whitespace), strings are lowercase, and there are no negative numbers or escape sequences. The regex dialect has no wildcard `.`, anchors or negated classes, and no shorthands such as `\d`.
- "Any other character" is infinitely many inputs, so generated negative tests try only a few likely offenders (space, underscore, dash, dot, at-sign, hash).

- After a top-100 check the PDA's probability-mass figure is a lower bound, and top-100 sampling is an approximation (greedy decoding is exact).
- The benchmark covers GPT-2 only. The Docker image was written but not built on the development machine, and there is no automated end-to-end browser test suite.

12. Coding

The complete source code, tests and data are in the project repository: https://github.com/MankiratBhatia/Grammar-Guard. This section reproduces the key routines, taken verbatim from the source files. Every function in the theory core has a docstring that names the step of the theory it implements; the docstrings are reproduced here because they carry the explanation.

Repository layout

Listing 2: Repository layout

```
grammarguard/
  core/        regex_parser, nfa, dfa, dot, algebra, elimination, counting, testgen,
               cfg, pda, schema, certificate            (standard library only)
  decoding/    token_index, generate, pda_generate, engine
  api/         FastAPI application: routers, schemas, limits, streaming
  eval/        benchmark suite, runner, statistics, report
frontend/      React + TypeScript + Vite + Tailwind application
tests/         unit, property-based, API and real-model tests
docs/          THEORY.md, API.md, VIVA.md, DEMO.md       eval/results/  benchmark data
```

Listing 3: `grammarguard/core/nfa.py`, `_ThompsonBuilder.build`. Thompson's construction by structural induction on the regex AST (Section 9.1).

```
def build(self, node: rp.Node) -> tuple[int, int]:
    """Thompson's construction by structural induction on the AST.

    Returns the (start, accept) pair of the fragment recognising L(node).
    """
    if isinstance(node, rp.Epsilon):
        s, f = self.new_state(), self.new_state()
        self.nfa.add_edge(s, None, f)
        return s, f
    if isinstance(node, rp.Literal):
        s, f = self.new_state(), self.new_state()
        self.nfa.add_edge(s, node.char, f)
        return s, f
    if isinstance(node, rp.CharClass):
        s, f = self.new_state(), self.new_state()
        for ch in node.chars:
            self.nfa.add_edge(s, ch, f)
        return s, f
    if isinstance(node, rp.Concat):
        # Glue R's accept to S's start with an epsilon edge.
        s1, f1 = self.build(node.left)
        s2, f2 = self.build(node.right)
        self.nfa.add_edge(f1, None, s2)
        return s1, f2
    if isinstance(node, rp.Union):
        # New start branches to both; both accepts join a new accept.
        s, f = self.new_state(), self.new_state()
        s1, f1 = self.build(node.left)
        s2, f2 = self.build(node.right)
        self.nfa.add_edge(s, None, s1)
        self.nfa.add_edge(s, None, s2)
        self.nfa.add_edge(f1, None, f)
        self.nfa.add_edge(f2, None, f)
        return s, f
    if isinstance(node, rp.Star):
        # Skip (zero copies) or enter; loop back from the end for more copies.
        s, f = self.new_state(), self.new_state()
        s1, f1 = self.build(node.child)
        self.nfa.add_edge(s, None, s1)
        self.nfa.add_edge(s, None, f)
        self.nfa.add_edge(f1, None, s1)
        self.nfa.add_edge(f1, None, f)
        return s, f
    raise TypeError(f"unknown AST node {node!r}")
```

Listing 4: `grammarguard/core/nfa.py`, `NFA.epsilon_closure`. Epsilon-closure as graph reachability over epsilon edges.

```
def epsilon_closure(self, states: set[int] | frozenset[int]) -> frozenset[int]:
    """Epsilon-closure E(S): all states reachable from S using only epsilon edges.

    Computed as graph reachability (depth-first search with an explicit stack)
    over the epsilon edges. S itself is always included (zero epsilon moves).
    """
    closure = set(states)
    stack = list(states)
    while stack:
        q = stack.pop()
        for r in self.delta.get((q, None), ()):
            if r not in closure:
                closure.add(r)
                stack.append(r)
    return frozenset(closure)
```

Listing 5: `grammarguard/core/dfa.py`, `subset_construction`. Subset construction: each DFA state is the epsilon-closure of a set of NFA states; only reachable subsets are built; a state cap guards against exponential blow-up.

```
def subset_construction(nfa: NFA, trace: list[dict[str, Any]] | None = None, max_states: int | None = None) -
> DFA:
    """Subset construction: each DFA state = epsilon-closure of a set of NFA states.

        q0'          = E({q0})
        delta'(S, a) = E(move(S, a))          for a in Sigma and a = OTHER
        F'           = { S : S contains an NFA accepting state }

    Only subsets reachable from q0' are generated (worklist algorithm), so we
    avoid building all 2^|Q| subsets. The empty subset, if reached, is the dead
    state; for a regex NFA it is always reached via OTHER.

    If `trace` is a list, one record per (subset, symbol) step is appended to it
    (used by the Learn page; it is the real algorithm, not a re-implementation).
    If `max_states` is given, raises StateLimitError instead of building a larger DFA.
    """
    symbols = sorted(nfa.alphabet) + [OTHER]
    start_set = nfa.epsilon_closure({nfa.start})
    ids: dict[frozenset[int], int] = {start_set: 0}
    worklist = [start_set]
    delta: dict[tuple[int, str], int] = {}
    while worklist:
        S = worklist.pop()
        for a in symbols:
            moved = nfa.move(S, a)
            U = nfa.epsilon_closure(moved)
            is_new = U not in ids
            if is_new:
                if max_states is not None and len(ids) >= max_states:
                    raise StateLimitError(max_states, "subset construction")
                ids[U] = len(ids)
                worklist.append(U)
            delta[(ids[S], a)] = ids[U]
            if trace is not None:
                trace.append(
                    {"from": sorted(S), "symbol": a, "move": sorted(moved), "closure": sorted(U), "new":
is_new}
                )
    accept = frozenset(i for S, i in ids.items() if S & nfa.accept)
    labels = {i: ("Ø" if not S else _fmt_set(S)) for S, i in ids.items()}
    return _renumber(ids[start_set], accept, delta, nfa.alphabet, labels)
```

Listing 6: `grammarguard/core/dfa.py`, `minimize_table`. Table-filling minimization (Myhill-Nerode): mark distinguishable pairs round by round, then merge the unmarked pairs.

```
def minimize_table(dfa: DFA, trace: list[dict[str, Any]] | None = None) -> DFA:
    """Table-filling minimization (Myhill-Nerode / Hopcroft's marking algorithm).

    1. Remove unreachable states.
    2. Mark every pair {p, q} with exactly one of p, q accepting
```

```
       (the empty string distinguishes them).
    3. Repeat until nothing changes: mark {p, q} if for some symbol a the pair
       {delta(p,a), delta(q,a)} is already marked (a longer distinguishing
       string exists).
    4. Unmarked pairs are equivalent; merge each equivalence class into one state.

    If `trace` is a list, each round's newly marked pairs (with the reason) are appended.
    """
    states = sorted(dfa.reachable_states())
    symbols = dfa.symbols()

    def key(p: int, q: int) -> tuple[int, int]:
        """Unordered pair as an ordered tuple (the table is triangular)."""
        return (p, q) if p < q else (q, p)

    marked: set[tuple[int, int]] = set()
    round0 = []
    for i, p in enumerate(states):
        for q in states[i + 1 :]:
            if (p in dfa.accept) != (q in dfa.accept):
                marked.add((p, q))
                round0.append({"pair": [p, q], "reason": "ε: one accepting, one not"})
    if trace is not None:
        trace.append({"round": 0, "marked": round0})

    changed, rnd = True, 0
    while changed:
        changed, rnd = False, rnd + 1
        newly = []
        for i, p in enumerate(states):
            for q in states[i + 1 :]:
                if (p, q) in marked:
                    continue
                for a in symbols:
                    p2, q2 = dfa.delta[(p, a)], dfa.delta[(q, a)]
                    if p2 != q2 and key(p2, q2) in marked:
                        marked.add((p, q))
                        changed = True
                        newly.append({"pair": [p, q], "symbol": a, "leads_to": list(key(p2, q2))})
                        break
        if trace is not None and newly:
            trace.append({"round": rnd, "marked": newly})

    # Each state's representative = smallest state equivalent to it.
    # (Unmarked = equivalent, and equivalence is transitive, so this is well defined.)
    rep: dict[int, int] = {}
    for q in states:
        rep[q] = next(p for p in states if p == q or (p < q and (p, q) not in marked))

    return _quotient(dfa, states, rep, trace)
```

Listing 7: `grammarguard/core/dfa.py`, `product`. Product construction, the proof of closure under intersection, union, difference and symmetric difference.

```
def product(d1: DFA, d2: DFA, mode: str = "intersection", limit: int | None = None) -> DFA:
    """Product construction: run d1 and d2 in parallel on the same input.

        Q = Q1 x Q2,  q0 = (q01, q02),  delta((p,q), a) = (delta1(p,a), delta2(q,a))
        F depends on `mode`: intersection (p in F1 and q in F2), union, difference,
        or symmetric difference.

    This is the proof that regular languages are closed under these operations.
    Sigma = Sigma1 U Sigma2; a character missing from one DFA's Sigma follows
    that DFA's OTHER edge. Only reachable pairs are built; `limit` caps how many
    (raises StateLimitError).
    """
    accepting = PRODUCT_MODES[mode]
    alphabet = d1.alphabet | d2.alphabet
    symbols = sorted(alphabet) + [OTHER]
    start = (d1.start, d2.start)
    seen = {start}
    queue = deque([start])
    delta: dict[tuple[tuple[int, int], str], tuple[int, int]] = {}
    while queue:
```

```
        p, q = queue.popleft()
        for a in symbols:
            nxt = (d1.step(p, a), d2.step(q, a))
            delta[((p, q), a)] = nxt
            if nxt not in seen:
                if limit is not None and len(seen) >= limit:
                    raise StateLimitError(limit, "product construction")
                seen.add(nxt)
                queue.append(nxt)
    accept = {pq for pq in seen if accepting(pq[0] in d1.accept, pq[1] in d2.accept)}
    labels = {pq: f"({pq[0]},{pq[1]})" for pq in seen}
    return _renumber(start, accept, delta, alphabet, labels)
```

Listing 8: `grammarguard/core/dfa.py`, `complement`. Complement: swap accepting and non-accepting states of a complete DFA.

```
def complement(dfa: DFA) -> DFA:
    """Closure under complement: same complete DFA, accepting and non-accepting swapped.

    Correct only because the DFA is complete over ALL characters (OTHER included):
    every string ends in exactly one state, so it is accepted by exactly one of M, M'.
    """
    accept = frozenset(q for q in dfa.states if q not in dfa.accept)
    return _renumber(dfa.start, accept, dfa.delta, dfa.alphabet, dfa.labels)
```

Listing 9: `grammarguard/core/counting.py`, `shortest_strings`. Shortest strings by a feasibility table and guided depth-first search (the corrected version, Section 13).

```
def shortest_strings(dfa: DFA, k: int, max_len: int = 64, include_other: bool = True) -> list[str]:
    """The first k accepted strings in shortlex order (by length, then by symbol order).

    OTHER is shown as a concrete character outside Sigma.

    A plain BFS over prefixes is exponential: for a dense language whose shortest
    match has length 8 over a 40-symbol alphabet it would queue ~40^7 prefixes.
    Instead we use a feasibility table
        can[r] = { q : some string of length exactly r leads from q to an accepting state }
    built by dynamic programming (can[0] = F; q is in can[r] if some symbol leads
    into can[r-1]). For each length L = 0, 1, 2, ... we enumerate the strings of
    length exactly L by depth-first search in symbol order, entering a state
    only if it is in can[remaining]. Every branch followed therefore ends in an
    accepted string, so the work is O(k * L * |symbols|) plus the table.
    """
    symbols = dfa.symbols() if include_other else sorted(dfa.alphabet)
    stand_in = other_char(dfa.alphabet)
    can: list[frozenset[int]] = [dfa.accept]
    found: list[str] = []

    def walk(q: int, remaining: int, prefix: list[str]) -> None:
        """DFS for strings of exactly `remaining` more symbols; stops once k are found."""
        if len(found) >= k:
            return
        if remaining == 0:
            found.append("".join(prefix))
            return
        for a in symbols:
            r = dfa.delta[(q, a)]
            if r in can[remaining - 1]:
                prefix.append(stand_in if a == OTHER else a)
                walk(r, remaining - 1, prefix)
                prefix.pop()
                if len(found) >= k:
                    return

    for length in range(max_len + 1):
        if length:
            prev = can[-1]
            can.append(frozenset(q for q in dfa.states if any(dfa.delta[(q, a)] in prev for a in symbols)))
        if dfa.start in can[length]:
            walk(dfa.start, length, [])
            if len(found) >= k:
```

```
                break
    return found
```

Listing 10: `grammarguard/core/cfg.py`, `Grammar.parse_table`. LL(1) parse table construction with conflict detection.

```
def parse_table(self) -> dict[tuple[str, str], tuple[Symbol, ...]]:
    """Build the LL(1) parse table M[A, c] -> right-hand side.

    Raises GrammarError listing every cell with two different productions
    (an LL(1) conflict)."""
    table: dict[tuple[str, str], tuple[Symbol, ...]] = {}
    conflicts = []
    for a, alts in self.productions.items():
        for rhs in alts:
            chars, nullable = self.first_of_seq(rhs)
            if nullable:
                chars = chars | self.follow[a]
            for c in chars:
                if (a, c) in table and table[(a, c)] != rhs:
                    conflicts.append(f"M[{a}, {c!r}]: {list(table[(a, c)])} vs {list(rhs)}")
                table[(a, c)] = rhs
    if conflicts:
        raise GrammarError("not LL(1):\n  " + "\n  ".join(sorted(conflicts)))
    return table
```

Listing 11: `grammarguard/core/pda.py`, `PDA.step`. One character of the deterministic pushdown automaton: expand nonterminals using the look-ahead, then match the terminal.

```
def step(self, stack: Stack, c: str) -> Stack | None:
    """Consume one character: expand nonterminals on top (using c as look-ahead),
    then match the terminal underneath. Returns the new stack, or None = reject."""
    work: list[Symbol] = list(stack)
    while work and isinstance(work[-1], str):
        nonterminal = work.pop()
        assert isinstance(nonterminal, str)
        rhs = self.table.get((nonterminal, c))
        if rhs is None:
            return None
        work.extend(reversed(rhs))
    if not work:  # input left over but nothing left to match
        return None
    top = work.pop()
    assert isinstance(top, Term)
    return tuple(work) if c in top.chars else None
```

Listing 12: `grammarguard/decoding/token_index.py`, `_walk`. Allowed tokens of one DFA state by a pruned depth-first search over the vocabulary trie and the DFA together.

```
def _walk(dfa: DFA, trie: VocabTrie, live: frozenset[int], q: int) -> dict[int, int]:
    """Allowed tokens of ONE state: {token id: next state}, by a pruned DFS over trie x DFA.

    Reading a character moves both the trie node and the DFA state; if the DFA
    state is not live the whole subtree below is pruned, since a non-live state
    can only lead to non-live states -- every token in that subtree would be
    rejected. Tokens ending at a visited node are allowed.
    """
    found: dict[int, int] = {}
    stack = [(0, q)]
    while stack:
        node, state = stack.pop()
        for ch, child in trie.children[node].items():
            r = dfa.step(state, ch)
            if r not in live:
                continue  # prune the whole subtree
            for tid in trie.tokens_at[child]:
                found[tid] = r
            stack.append((child, r))
    return found
```

Listing 13: `grammarguard/decoding/generate.py`, `DFAMask.prepare`. Masking the logits for the current state; end-of-text only in accepting states.

```
def prepare(self, logits: torch.Tensor):
    """Mask logits for the current state (None on a dead end); remember what telemetry needs."""
    q = self.state
    if q not in self._ids:
        self._ids[q] = torch.tensor(self.index.allowed_for(q), dtype=torch.long)
    allowed = self._ids[q]
    accepting = q in self.dfa.accept
    if len(allowed) == 0 and not accepting:
        return None
    if accepting:  # EOS is legal exactly in accepting states
        allowed = torch.cat([allowed, torch.tensor([self.eos_id])])
    masked = torch.full_like(logits, float("-inf"))
    masked[allowed] = logits[allowed]
    self._pending = (logits, allowed)
    return masked, len(allowed)
```

Listing 14: `grammarguard/decoding/generate.py`, `_telemetry`. Constraint pressure: probability mass kept, cost in bits, entropy before and after, and whether the model's first choice was allowed.

```
def _telemetry(logits: torch.Tensor, allowed: torch.Tensor, chosen: int, tokenizer) -> Telemetry:
    """Compute the pressure numbers from raw logits and the allowed set (all in log space)."""
    ln2 = math.log(2)
    logp = torch.log_softmax(logits.double(), dim=-1)
    log_mass = torch.logsumexp(logp[allowed], dim=0)
    mass = float(torch.exp(log_mass))
    preferred = int(torch.argmax(logits))
    h_before = float(-(torch.exp(logp) * logp).sum()) / ln2
    cond = logp[allowed] - log_mass  # log of the renormalised distribution on A
    h_after = float(-(torch.exp(cond) * cond).sum()) / ln2
    return Telemetry(
        mass_kept=mass,
        pressure=1.0 - mass,
        cost_bits=float(-log_mass) / ln2,
        preferred=tokenizer.decode([preferred]),
        preferred_legal=bool((allowed == preferred).any()),
        p_chosen=float(torch.exp(logp[chosen])),
        entropy_before=h_before,
        entropy_after=h_after,
    )
```

Listing 15: `grammarguard/core/certificate.py`, `verify_dfa_certificate`. Replaying a certificate against an automaton rebuilt from the constraint text.

```
def verify_dfa_certificate(cert: Certificate, dfa: DFA) -> Verification:
    """Replay the recorded run against a DFA the VERIFIER built from `cert.constraint`."""

    def bad(reason: str) -> Verification:
        return Verification(False, False, reason)

    if cert.kind != "dfa":
        return bad(f"not a DFA certificate (kind={cert.kind!r})")
    if cert.fingerprint != dfa_fingerprint(dfa):
        return bad("fingerprint mismatch: the certificate was made with a different automaton")
    if len(cert.trace) != len(cert.text) + 1:
        return bad(f"trace has {len(cert.trace)} states, expected {len(cert.text) + 1}")
    if cert.trace[0] != dfa.start:
        return bad(f"run starts in q{cert.trace[0]}, the start state is q{dfa.start}")
    for i, ch in enumerate(cert.text):
        expected = dfa.step(cert.trace[i], ch)
        if cert.trace[i + 1] != expected:
            return bad(
                f"step {i + 1}: δ(q{cert.trace[i]}, {ch!r}) = q{expected}, certificate says q{cert.trace[i +
1]}"
            )
    accepted = cert.trace[-1] in dfa.accept
    if cert.accepted != accepted:
        return bad(
            f"final state q{cert.trace[-1]} is {'accepting' if accepted else 'not accepting'}, certificate
disagrees"
```

```
        )
    if cert.token_ends != sorted(cert.token_ends) or any(not 0 < e <= len(cert.text) for e in
cert.token_ends):
        return bad("token boundaries are not increasing offsets inside the text")
    return Verification(True, accepted, f"all {len(cert.text)} transitions check out")
```

13. Screenshots of the Errors Segments

This section shows two kinds of errors. **Part A** shows errors that a *user* can cause and that the application detects and reports (all captured from the running application). **Part B** shows errors met by the *developers* while building the project. In Part B the first three were reproduced with the original code for this report, using exactly the failing logic; the last two (tooling errors that cannot be triggered again on the finished project) are transcribed from the development session and are marked as such in the captions.

Part A. Errors reported by the application

Fig.	Error	Cause	Detected by
15	Syntax error	Input `ab(c`: the opening bracket is never closed	Recursive-descent parser: position 2
16	Unsupported syntax	Input `a.b`: the wildcard `.` is deliberately not supported	Tokenizer: position 1, with advice to escape the dot
17	Unknown name in a rule	Expression `email & nonsense`: `nonsense` is not defined	Algebra parser: position 8
18	Pattern too complex	Pattern `(a|b)*a(a|b){20}` would need about 2^21 DFA states	Subset construction state cap (5,000), refused in 0.03 s
19	Invalid schema	The schema text is not valid JSON	Interface check before anything is sent; Generate is disabled

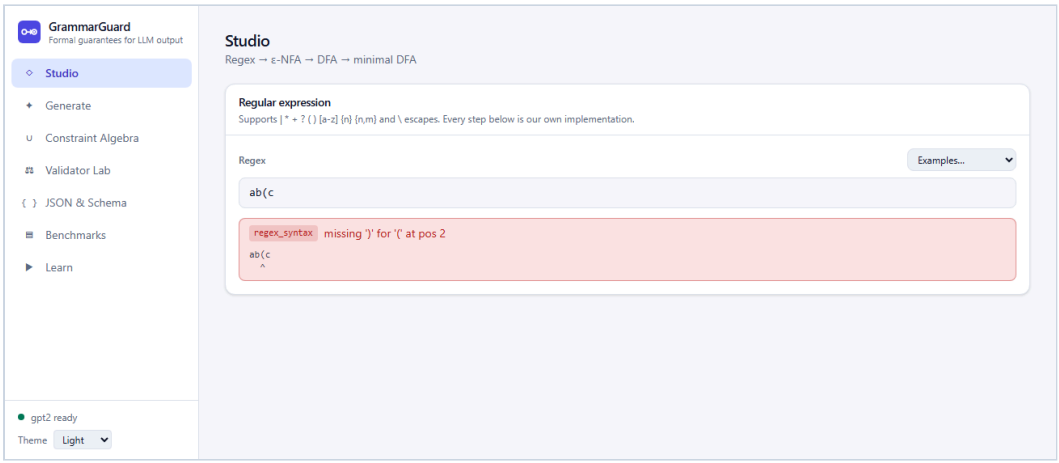


Figure 15: Error 1: a missing closing bracket is reported with the error code, the message and a caret under the position.

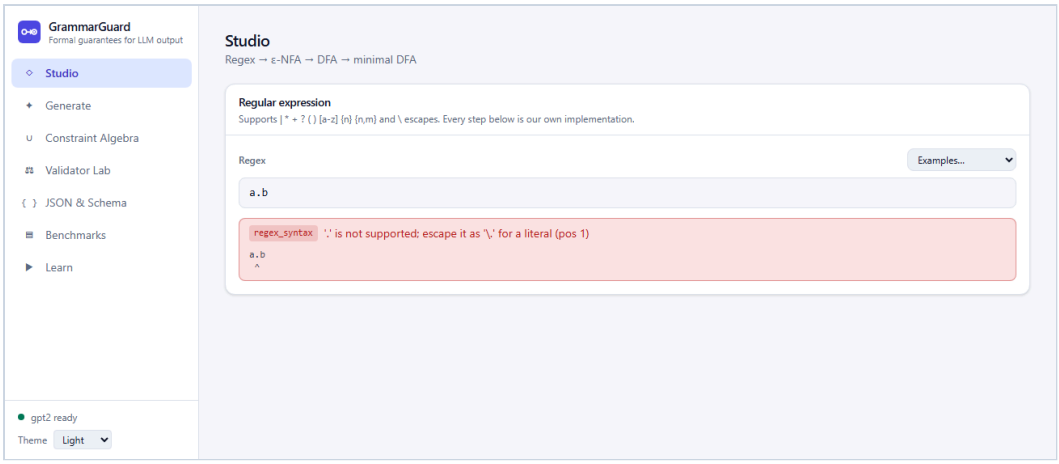


Figure 16: Error 2: the unsupported wildcard is rejected and the message says how to write a literal dot.

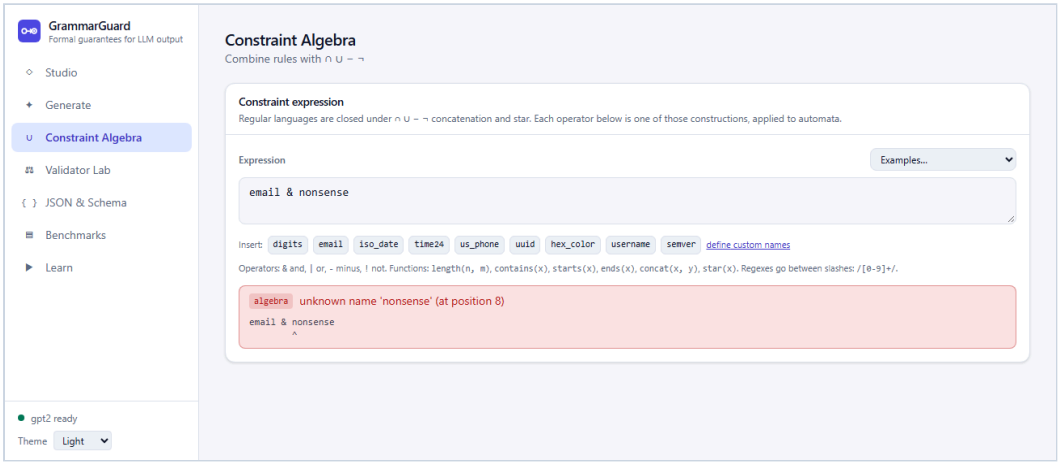


Figure 17: Error 3: an unknown name in a constraint expression, with the position of the name.

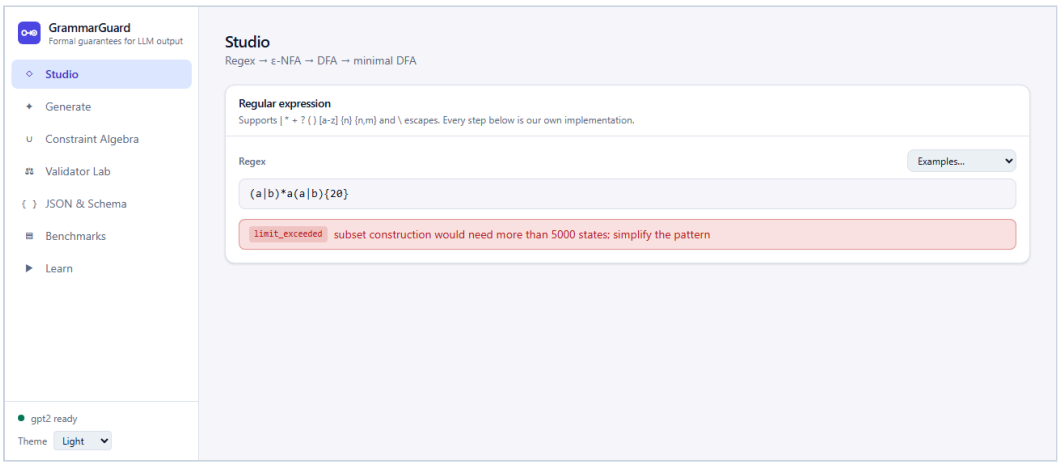


Figure 18: Error 4: a pattern whose automaton would be exponentially large is refused with a limit error instead of exhausting the server.

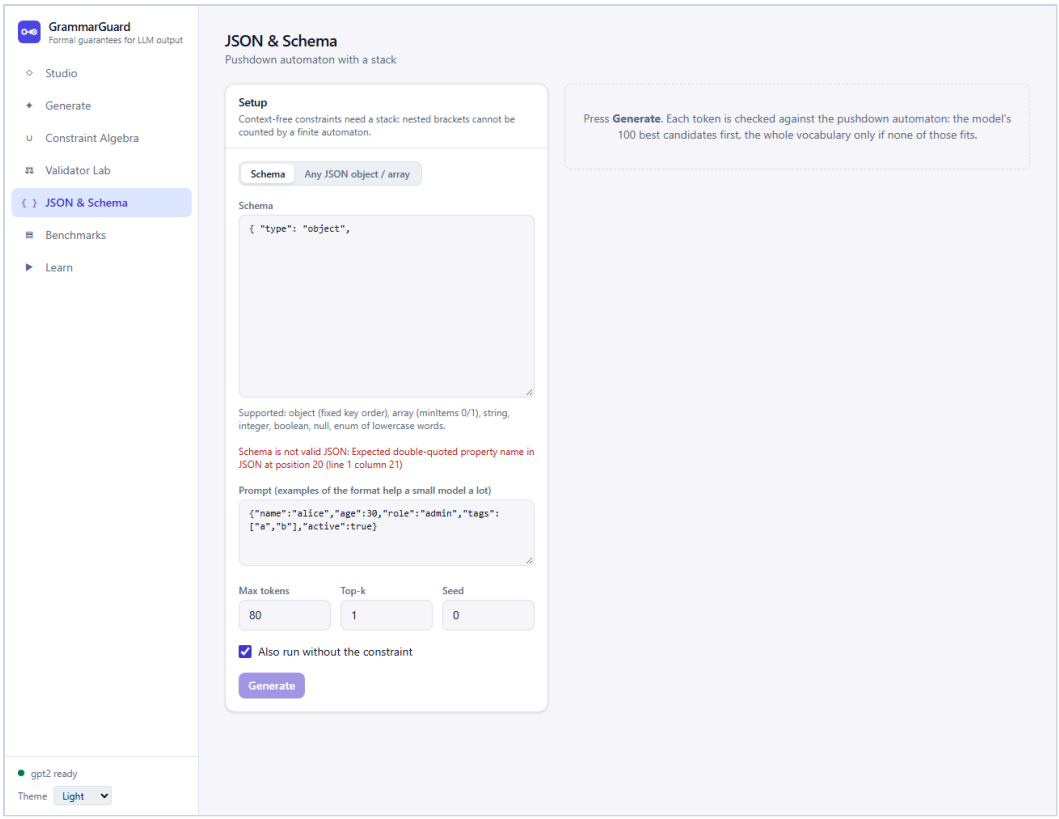


Figure 19: Error 5: an invalid schema is reported at once and the Generate button is disabled.

Part B. Errors met during development

Fig.	Error	Cause
20	LL(1) conflict found by the project's own grammar checker (reproduced)	In the array rule of the schema compiler both alternatives began with `[`, so the table cell M[V, '['] received two productions
21	Memory exhaustion in `shortest_strings` (reproduced under a 2 GB cap)	A breadth-first search over every viable prefix: for a dense language whose shortest match has 8 characters the queue quadruples at every level (1,025, 4,021, 16,005, ... prefixes). Without a cap this crashed the development virtual machine several times
22	Floating-point error in the confidence interval (reproduced)	For 100 successes in 100 trials the upper Wilson bound is mathematically 1 but was computed as 0.9999999999999999, which failed the test that the interval contains the observed rate
23	npm dependency resolver crash (transcribed from the session)	npm 10.8 failed inside its peer-dependency resolver for this set of packages
24	Type generator crash (transcribed from the session)	The OpenAPI type generator needs the TypeScript compiler's JavaScript API, which TypeScript 7 no longer provides

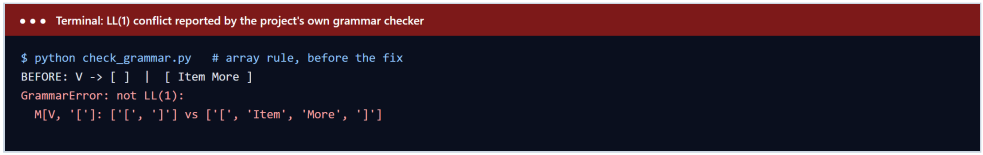


Figure 20: Development error 1: LL(1) conflict (reproduced).

Terminal: the original search exhausts memory (reproduced under a 2 GB cap)

```
$ python original_shortest_strings.py   # memory capped at 2 GB for safety
constraint: email & !contains(/spam/) & length(8, 30)   (649 DFA states)
original shortest_strings(): breadth-first search over every viable prefix
  queue holds      1,025 prefixes  (length of current prefix: 1)
  queue holds      4,021 prefixes  (length of current prefix: 2)
  queue holds     16,005 prefixes  (length of current prefix: 2)
  queue holds     64,017 prefixes  (length of current prefix: 3)
  queue holds    256,008 prefixes  (length of current prefix: 3)
  queue holds  1,024,036 prefixes  (length of current prefix: 3)
  queue holds  4,096,022 prefixes  (length of current prefix: 4)
MemoryError after 6.0 s  (limit 2 GB)
```

Figure 21: Development error 2: memory exhaustion of the original shortest-strings search (reproduced under a 2 GB limit).

Terminal: floating-point error at the boundary of the confidence interval

```
$ python wilson_check.py   # original interval formula
original wilson(100, 100) = (0.9630051925239981, 0.9999999999999999)
Traceback (most recent call last):
  File "wilson_check.py", line 11, in <module>
    assert 0 <= lo <= k / n <= hi <= 1, f"interval {lo, hi} does not contain the observed rate {k / n}"
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: interval (0.9630051925239981, 0.9999999999999999) does not contain the observed rate 1.0
```

Figure 22: Development error 3: the interval does not contain the observed rate (reproduced).

Terminal: npm's dependency resolver crashes

```
$ npm install --no-fund --no-audit -D vite
npm error Cannot read properties of null (reading 'edgesOut')
npm error A complete log of this run can be found in: ~/.npm/_logs/2026-10-28T19_18_36_596Z-debug-0.log
# from that log:  TypeError: Cannot read properties of null (reading 'edgesOut')
#                 at #loadPeerSet (npm/node_modules/@npmcli/arborist/...)
# (transcribed from the development session)
```

Figure 23: Development error 4: npm resolver crash (transcribed from the development session).

Terminal: type generator fails with TypeScript 7

```
$ npx openapi-typescript src/api/openapi.json -o src/api/schema.d.ts
const BOOLEAN = ts.factory.createKeywordTypeNode(ts.SyntaxKind.BooleanKeyword);
                           ^
TypeError: Cannot read properties of undefined (reading 'createKeywordTypeNode')
    at file:///.../node_modules/openapi-typescript/dist/lib/ts.mjs:11:28
Node.js v20.20.2
# (transcribed from the development session)
```

Figure 24: Development error 5: type generator crash with TypeScript 7 (transcribed from the development session).

Other issues found and resolved (described in the project's decision log): the first benchmark used prompts that ended mid-item and a harsh retry baseline (fixed, benchmark re-run); one disconnect test passed without generating anything because its fake language was too long (tightened to assert that generation started and stopped early); converting a 649-state automaton back to a regex needed 13.7 s even to give up (the service now attempts it only for automata of at most 40 states); and the application contacted the model hub at start-up even with cached weights (the Docker image now sets offline mode).

14. Screenshots of the Error Recovered Segments

Each error of Section 13 is shown again after it was corrected or recovered from, in the same order.

Part A. Recovery in the application

Fig.	Recovery	Result
25	The user closes the bracket: `ab(c)`	The pipeline compiles and the three automata are drawn
26	The user escapes the dot: `a\.b`	A literal dot is matched and the automata are drawn
27	The user replaces the unknown name: `email & length(8, 30)`	The expression is evaluated; size, examples and construction steps appear
28	The user lowers the bound: `(a|b)*a(a|b){6}`	The pattern compiles (its minimal DFA has 129 states)
29	The user supplies a valid schema	The error message disappears and Generate is enabled again

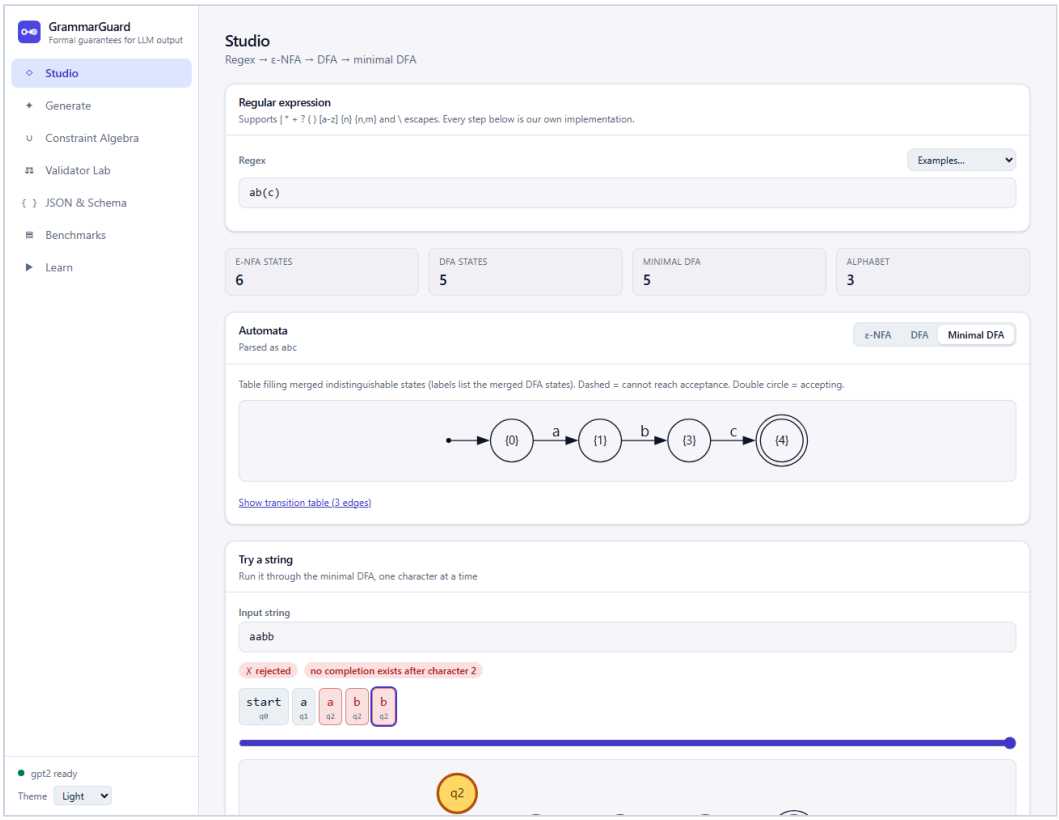


Figure 25: Recovery 1: after fixing the bracket the regex compiles.

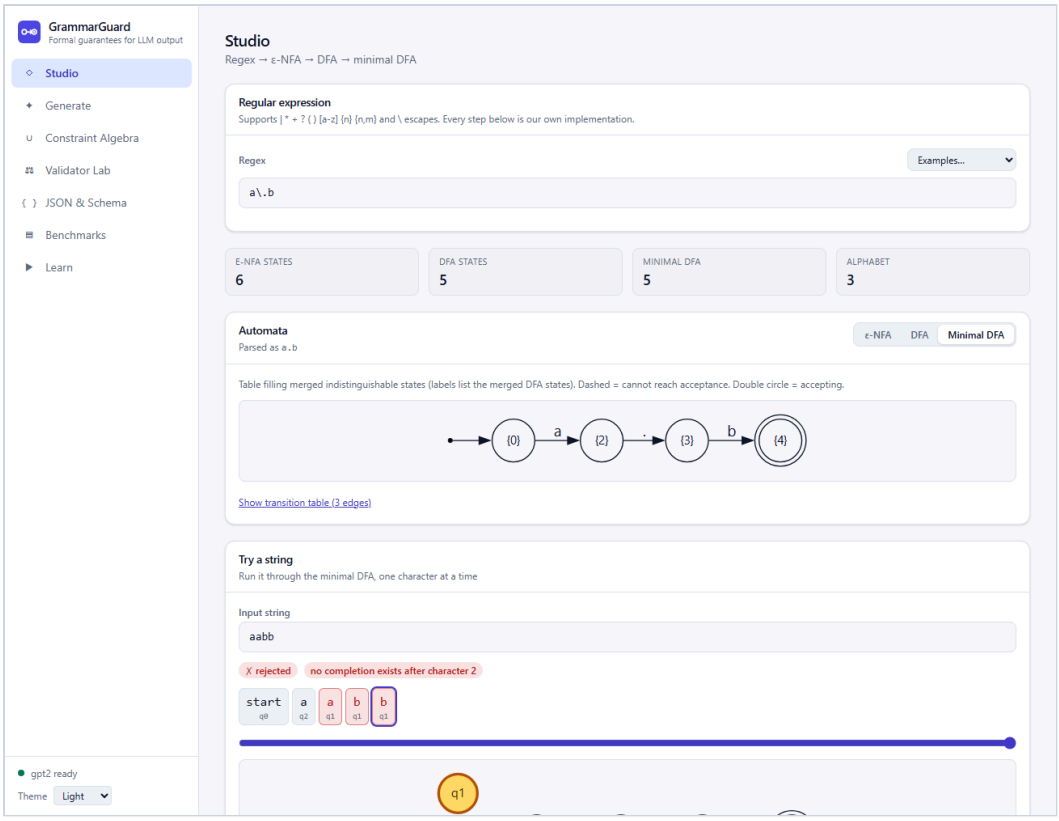


Figure 26: Recovery 2: the escaped dot is accepted.

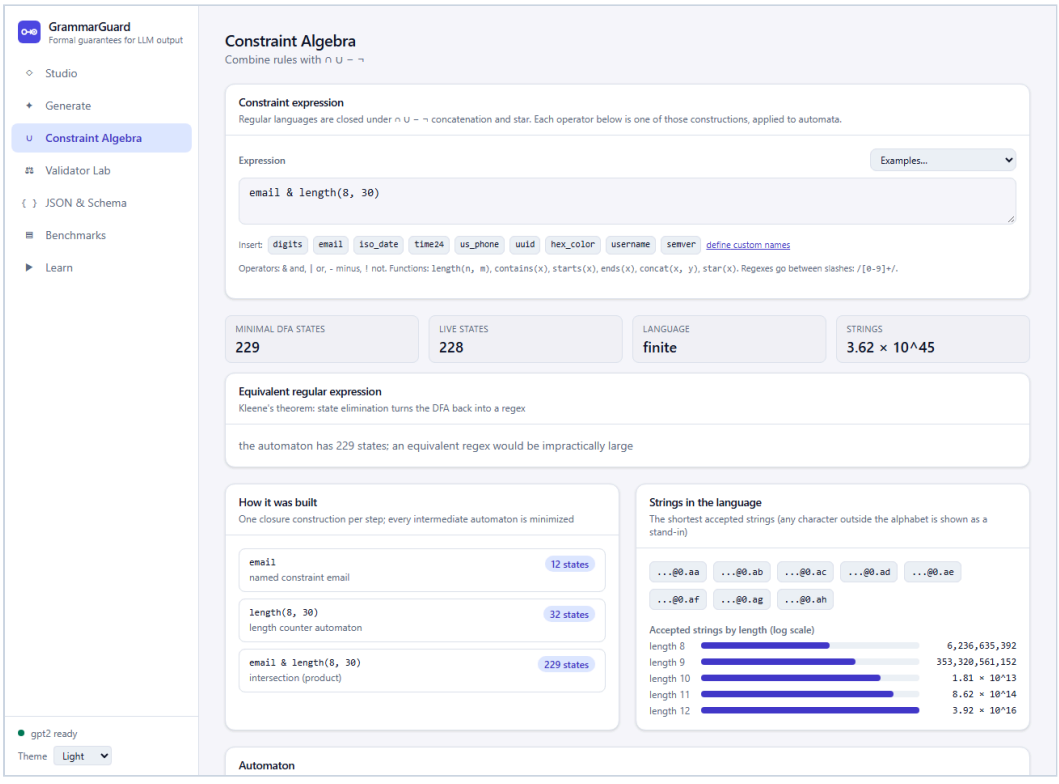


Figure 27: Recovery 3: the corrected expression is evaluated.

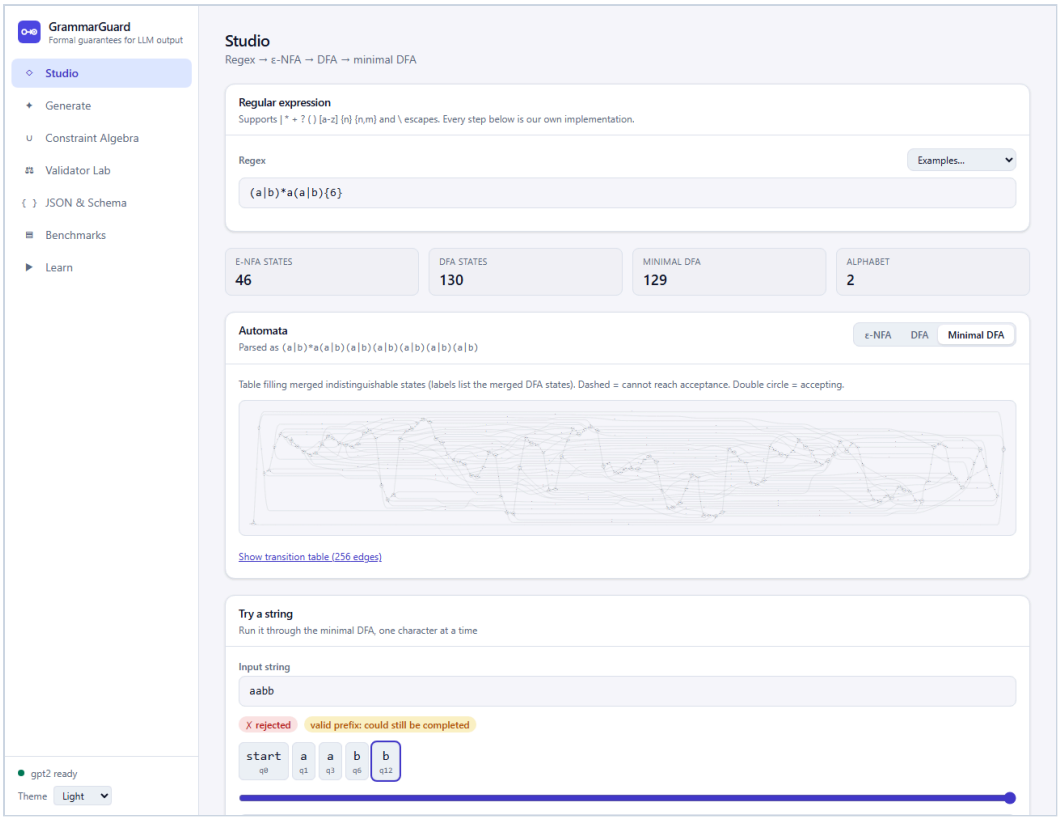


Figure 28: Recovery 4: a smaller bound gives a manageable automaton.

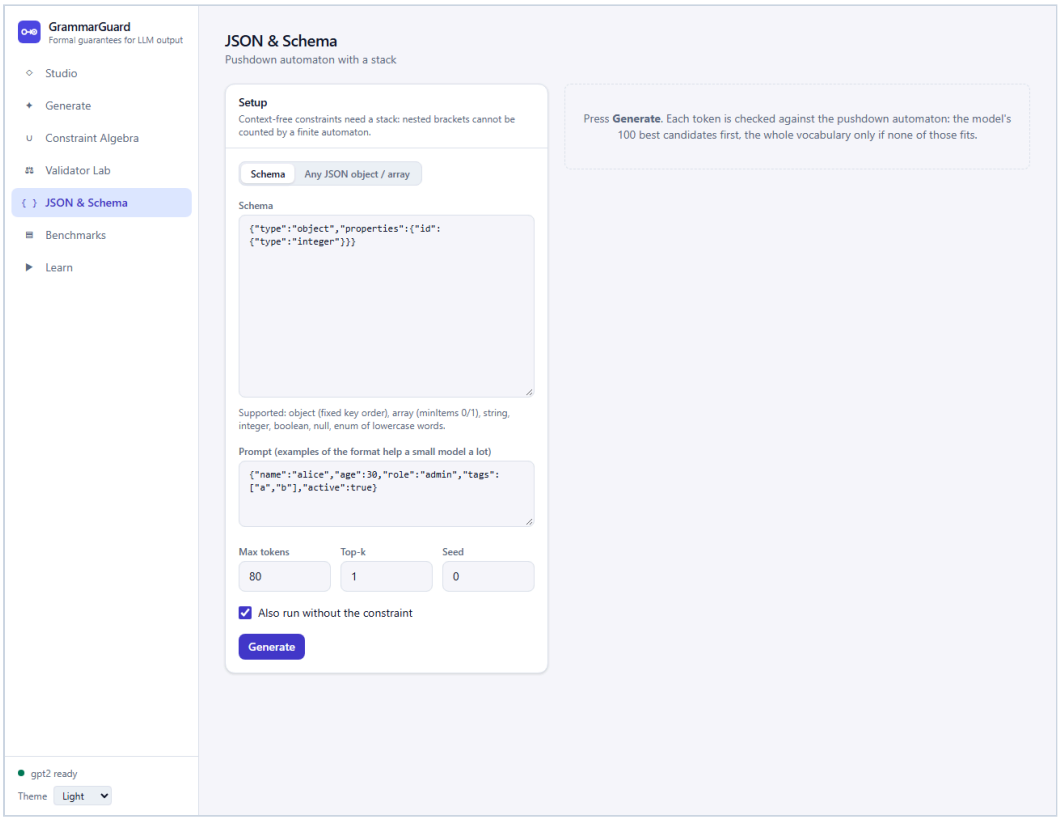


Figure 29: Recovery 5: a valid schema re-enables generation.

Part B. Corrections made during development

Fig.	Fix	Verification
30	Left-factoring: `V -> [ Body, Body -> ] | Item More ]`; the compiler now always builds the parse table so a conflict cannot slip through	The table builds with 24 cells and no conflicts
31	A feasibility table can[r] and a depth-first search that follows only branches guaranteed to end in a result	8 strings in about 8 ms for the same query; regression tests compare the output with brute force and check timing
32	Endpoints made exact at 0 successes and at n successes	The interval is (0.963..., 1.0) and the test passes
33	`legacy-peer-deps=true` in the project's `.npmrc`, with the peer ranges checked by hand and one peer dependency listed explicitly	86 packages installed; all frontend tests pass
34	TypeScript pinned to version 5.9.3	Types are generated from the backend schema again

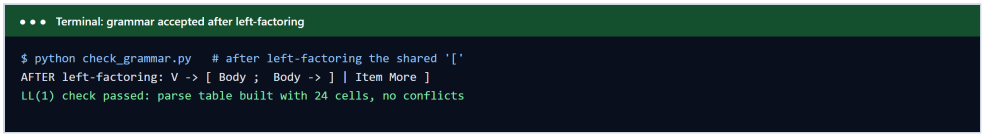


Figure 30: Correction 1: the grammar after left-factoring is LL(1).

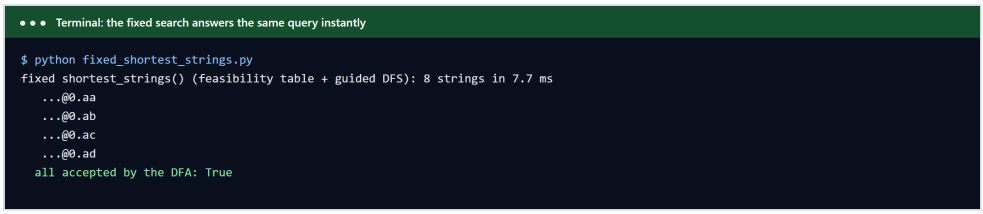


Figure 31: Correction 2: the corrected search answers instantly and every string is accepted.

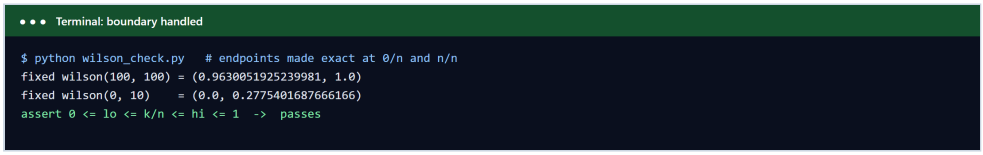


Figure 32: Correction 3: the boundary case is exact.

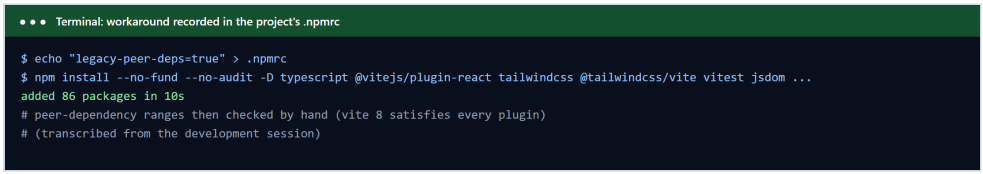


Figure 33: Correction 4: dependencies install with the recorded workaround (transcribed from the development session).

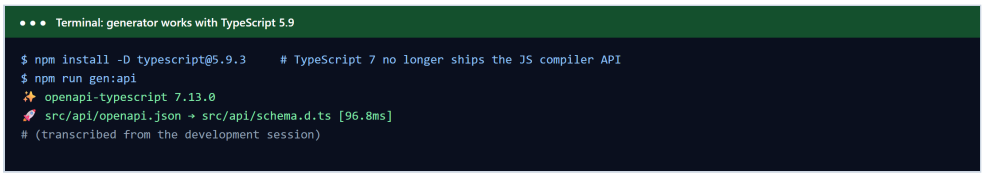


Figure 34: Correction 5: the type generator works with TypeScript 5.9 (transcribed from the development session).

15. Possible Future Work

- **Full context-free languages.** Add Chomsky and Greibach normal forms and the CYK algorithm, and nondeterministic or generalised pushdown parsing, so that grammars that are not LL(1) can be used; the current system supports deterministic context-free languages only.
- **More of the course.** Pumping-lemma tools (searching for a pumping decomposition or a counter-example), linear bounded automata and context-sensitive languages, a Turing-machine simulator, and demonstrations of undecidability (for example the Post correspondence problem).
- **Richer specifications.** Symbolic alphabets so that a wildcard `.` and negated classes can be expressed, whitespace, escape sequences and negative numbers in JSON, and a fuller JSON-schema subset (optional properties, bounded arrays, numeric ranges).
- **Stronger models and wider evaluation.** Repeat the benchmark with larger models and several model families, add structured-output tasks beyond formats (for example code in a grammar), and add a user study of the validator tools.
- **Performance.** Persist the lazy index to disk, support GPU inference and batching, reduce per-token overhead further, and look ahead over several tokens (speculative decoding with automaton checks).
- **Deployment.** Build and publish the Docker image, add authentication and rate limiting for public hosting, and add an automated end-to-end browser test suite.
- **Education.** Add more step players (Thompson construction, product construction, LL(1) table construction) and exercises with automatic feedback.

16. Conclusion

GrammarGuard shows that the central ideas of the Theory of Computation course, regular expressions, nondeterministic and deterministic automata, minimization, closure properties, decision problems, context-free grammars and pushdown automata, are not only theory: implemented carefully, they let a language model's output be *guaranteed* to satisfy a formal specification. The guarantee follows from a short argument (every allowed token keeps the automaton in a live state, and the model may stop only in an accepting state) and was confirmed empirically: in 288 benchmark runs over 18 validators the constrained output was valid every time, while plain GPT-2 was valid in none, and in 11% of runs even when a valid prefix was extracted from its output.

Because the automata were built from scratch, the project goes beyond constraining a model. It combines rules with closure operations, compares versions of a validator with shortest counterexamples, generates test suites with full transition coverage, counts and fairly samples languages, converts automata back to regular expressions, and issues certificates that anyone can verify without the model. The constraint-pressure metric turns the abstract notion of the specification overriding the model into a number of bits with a precise information-theoretic meaning.

The engineering was as instructive as the theory: a naive search that looked correct exhausted memory on realistic constraints, a benchmark that looked convincing was unfair until its raw outputs were read, and a trie that is far faster for restrictive constraints is slower for permissive ones. Each was found, understood and corrected, and the corrections are recorded in tests and documentation. The result is a documented, tested and open-source system, with a service API and a web interface, in which every step from regular expression to guaranteed output can be inspected.

17. References

References marked ★ are the **base papers** of this project; they are shaded and summarised in Section 17.1. The **base links** (open-source implementations, model card and documentation the work builds on) are listed in Section 17.3. Numbers in square brackets in the text refer to this list. Every title, author list, year and DOI below was checked against the publisher or arXiv record.

17.1 Base papers and their role in this project

Base paper	How it is used
★ [1]	Regex to finite-state machine to an index of allowed vocabulary tokens: the design of our token index.
★ [2]	Grammar-constrained decoding of structured outputs: basis of our grammar (LL(1)) decoding.
★ [3]	Automata-based constraints (DFA and pushdown) with token-level lookup: closest to our approach.
★ [7]	Regular expression to epsilon-NFA construction (nfa.py).
★ [8]	Subset construction, NFA to DFA (dfa.py).
★ [9]	Regular expressions equal finite automata; DFA to regex by state elimination (elimination.py).
★ [10]	Myhill-Nerode theorem: the basis of DFA minimization.
★ [11]	Myhill-Nerode theorem: the basis of DFA minimization.
★ [14]	Top-down (LL) parsing: basis of the LL(1) table and the pushdown automaton (cfg.py, pda.py).
★ [16]	Primary textbook for the automata and grammar algorithms.

17.2 Complete reference list

A. Base papers and related work on constrained decoding

[1] ★ B. T. Willard and R. Louf, "Efficient Guided Generation for Large Language Models," arXiv:2307.09702, 2023. https://arxiv.org/abs/2307.09702

[2] ★ S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning," in Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP), Main Conference, 2023; arXiv:2305.13971. https://arxiv.org/abs/2305.13971

[3] ★ T. Koo, F. Liu, and L. He, "Automata-based constraints for language model decoding," in Proc. Conf. on Language Modeling (COLM), 2024; arXiv:2407.08103. https://arxiv.org/abs/2407.08103

[4] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao, and T. Chen, "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models," in Proc. MLSys, 2025; arXiv:2411.15100. https://arxiv.org/abs/2411.15100

[5] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," in Proc. EMNLP, 2021; arXiv:2109.05093. https://arxiv.org/abs/2109.05093

[6] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation," arXiv:2403.06988, 2024. https://arxiv.org/abs/2403.06988

B. Base papers on automata, formal languages and parsing

[7] ★ K. Thompson, "Programming Techniques: Regular expression search algorithm," Communications of the ACM, vol. 11, pp. 419-422, 1968. https://doi.org/10.1145/363347.363387

[8] ★ M. O. Rabin and D. Scott, "Finite Automata and Their Decision Problems," IBM Journal of Research and Development, vol. 3, pp. 114-125, 1959. https://doi.org/10.1147/rd.32.0114

[9] ★ S. C. Kleene, "Representation of events in nerve nets and finite automata," in Automata Studies, C. E. Shannon and J. McCarthy, Eds. Princeton University Press, 1956, pp. 3-41.

[10] ★ J. Myhill, "Finite automata and the representation of events," WADD Technical Report 57-624, Wright Air Development Division, Wright-Patterson Air Force Base, 1957.

[11] ★ A. Nerode, "Linear automaton transformations," Proceedings of the American Mathematical Society, vol. 9, pp. 541-544, 1958. https://doi.org/10.1090/S0002-9939-1958-0135681-9

[12] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies, C. E. Shannon and J. McCarthy, Eds. Princeton University Press, 1956, pp. 129-153.

[13] N. Chomsky, "On certain formal properties of grammars," Information and Control, vol. 2, pp. 137-167, 1959. https://doi.org/10.1016/S0019-9958(59)90362-6

[14] ★ P. M. Lewis II and R. E. Stearns, "Syntax-directed transduction," Journal of the ACM, vol. 15, pp. 465-488, 1968. https://doi.org/10.1145/321466.321477

[15] D. J. Rosenkrantz and R. E. Stearns, "Properties of deterministic top-down grammars," Information and Control, vol. 17, pp. 226-256, 1970. https://doi.org/10.1016/S0019-9958(70)90446-8

C. Textbooks

[16] ★ J. E. Hopcroft, R. Motwani, and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed. Pearson, 2006.

[17] M. Sipser, Introduction to the Theory of Computation, 3rd ed. Cengage Learning, 2012.

[18] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson, 2006.

D. Language models

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[20] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems 30, 2017; arXiv:1706.03762. https://arxiv.org/abs/1706.03762

[21] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proc. 54th Annual Meeting of the ACL, 2016; arXiv:1508.07909. https://arxiv.org/abs/1508.07909

[22] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in Proc. EMNLP 2020: System Demonstrations, pp. 38-45, 2020. https://doi.org/10.18653/v1/2020.emnlp-demos.6

E. Statistics, testing and tooling

[23] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, pp. 209-212, 1927. https://doi.org/10.1080/01621459.1927.10502953

[24] S. Kullback and R. A. Leibler, "On Information and Sufficiency," The Annals of Mathematical Statistics, vol. 22, pp. 79-86, 1951. https://doi.org/10.1214/aoms/1177729694

[25] D. R. MacIver, Z. Hatfield-Dodds, et al., "Hypothesis: A new approach to property-based testing," Journal of Open Source Software, vol. 4, p. 1891, 2019. https://doi.org/10.21105/joss.01891

[26] R. Cox, "Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...)," 2007. https://swtch.com/~rsc/regexp/regexp1.html

[27] E. R. Gansner and S. C. North, "An open graph visualization system and its applications to software engineering," Software: Practice and Experience, vol. 30, pp. 1203-1233, 2000. https://doi.org/10.1002/1097-024X(200009)30:11<1203::AID-SPE338>3.0.CO;2-N

17.3 Base links and software documentation

Resource	Link	Role
Project repository (source code, tests, benchmark data)	https://github.com/MankiratBhatia/Grammar-Guard	This project
★ **Outlines: structured generation library**	https://github.com/dottxt-ai/outlines	Base link: open-source implementation of the regex-to-automaton token index described in [1]

Resource	Link	Role
★ **llama.cpp grammar-constrained sampling (GBNF)**	https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md	Base link: grammar-constrained decoding in a widely used inference engine
★ **GPT-2 model card (Hugging Face)**	https://huggingface.co/openai-community/gpt2	Base link: the language model used in this project
Hugging Face Transformers documentation	https://huggingface.co/docs/transformers	Library used to load GPT-2
PyTorch	https://pytorch.org	Tensor library (CPU build)
FastAPI	https://fastapi.tiangolo.com	Web framework for the backend
Pydantic	https://docs.pydantic.dev	Request and response models
React	https://react.dev	Frontend library
Vite	https://vite.dev	Frontend build tool
Tailwind CSS	https://tailwindcss.com	Styling
TypeScript	https://www.typescriptlang.org	Typed frontend language
Viz.js (Graphviz compiled to WebAssembly)	https://github.com/mdaines/viz-js	Draws the automata in the browser
Graphviz	https://graphviz.org	DOT graph description language
Python re module documentation	https://docs.python.org/3/library/re.html	Used only as an independent test oracle
Hypothesis	https://hypothesis.readthedocs.io	Property-based testing
pytest	https://docs.pytest.org	Python test runner
Server-Sent Events (MDN)	https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events	Token streaming protocol
JSON: RFC 8259	https://www.rfc-editor.org/rfc/rfc8259	Specification of the JSON format (the project supports a subset)
Docker documentation	https://docs.docker.com	Container packaging